

TOPICAL REVIEW

A Review of Reinforcement Learning for Fixed-Wing Aircraft Control Tasks

DAVID J. RICHTER^{1,2}, RICARDO A. CALIX², AND KYUNGBAEK KIM¹, (Member, IEEE)

¹Department of Artificial Intelligence Convergence, Chonnam National University, Gwangju 61186, South Korea

²College of Technology, Purdue University Northwest, Hammond, IN 46323, USA

Corresponding author: Kyungbaek Kim (kyungbaekkim@jnu.ac.kr)

This work was supported in part by Korea Institute of Planning and Evaluation for Technology in Food, Agriculture and Forestry (IPET) through the Agriculture and Food Convergence Technologies Program for Research Manpower Development, funded by the Ministry of Agriculture, Food and Rural Affairs (MAFRA), under Project RS-2024-00397026, 33%; in part by the Innovative Human Resource Development for Local Intellectualization Program through the Institute of Information and Communications Technology Planning and Evaluation (IITP) Grant funded by Korea Government (MSIT) under Grant IITP-2024-00156287, 33%; and in part by the IITP through the Artificial Intelligence Convergence Innovation Human Resources Development Grant funded by Korea Government (MSIT) under Grant IITP-2023-RS-2023-00256629, 34%.

ABSTRACT Reinforcement learning (RL) has seen an uptick in research interest in recent years, with many papers published in a plethora of different fields, topics and applications. A lot of that can be attributed to the recent advancements in machine learning (ML) and deep learning (DL) as a whole, the power of deep neural networks and the incorporation of them into reinforcement learning algorithms and techniques. This marriage of deep neural networks and traditional reinforcement learning methods has enabled reinforcement learning algorithms to achieve incredible results in use-cases that were previously deemed to be too complex. One of these highly complex scenarios is the realm of flight. In the last few years the number of papers published in this field has increased dramatically, but there is a fairly big divide between two major categories (quadcopters vs. fixed-wing aircraft). There is published work focusing on reinforcement learning using quadcopters, also known as drones, and there is published work focusing on fixed-wing aircraft, the more traditional aircraft, commonly referred to as airplanes. This paper focuses on the latter and will give an overview over the current state of reinforcement learning in the field of fixed-wing aviation. It will look into available toolkits usable for research as well as give an overview of different fields and tasks within reinforcement learning for fixed-wing aircraft control tasks. It contributes by providing detailed information about the methodologies used in each paper, while pointing out each paper's contributions as well. A total of 48 papers on fixed-wing RL and airplane control are discussed. With the main focus lying on papers published post 2015, after the publication of Mnih et al.'s impactful deep RL paper.

INDEX TERMS Airplanes, artificial intelligence, autonomous flight, aviation, deep learning, deep reinforcement learning, fixed-wing aircraft, flight control, machine learning, reinforcement learning.

I. INTRODUCTION

Reinforcement learning (RL) has been studied for many years already, but it has just seen its biggest jump in popularity and applicability in recent years. A lot of that can be attributed to Mnih et al. [1], [2], who introduced the deep Q-network (DQN) methodology, which successfully implemented deep neural networks (DNN) into the traditional reinforcement

learning workflow. Before DQNs, a lot of the RL landscape was shaped by look-up table approaches, where the policy was simply saved in such a table. This obviously meant that RL at that point in time was rather limited, since otherwise the table would be exorbitantly large (because table based RL does not generalize). One very popular algorithm at the time was Q-learning (QL) [3]. An early successful experiment of using neural networks (NN) with RL was done by Tesauro et al. [4], but it was found to not be generally applicable to other fields. Mnih's DQN managed to work

The associate editor coordinating the review of this manuscript and approving it for publication was Zhenbao Liu¹.

in a wide variety of fields and led to somewhat of an RL boom. Many works were conducted in many different fields and areas, algorithms were improved, new ones developed and toolkits and code bases were created to aid researchers in setting up their respective work. Another big leap was done when Silver et al. [5], [6] developed and published AlphaGo. This RL model managed to beat the world champion of the game Go (Lee Sedol) in a series of games. This achievement was in the news all around the world and put RL in the spotlight. The game of Go was considered to be too difficult to solve for another decade by experts at the time, showing off the great advances RL was making. Ever since then, RL has been used in a large number of fields. Be it autonomous driving [7], video games [8], networking [9], traditional games [10], robotics [11] and many other tasks, among which are also flight tasks [12], [13], [14]. Reinforcement learning has also received attention in various different fields of transportation, including driving, maritime navigation, and rail transportation, among others [15]. One difference between driving, rail transportation, and maritime navigation, and flying is that in flight one must consider all 6 degrees of freedom (DoF), whereas, mostly, only 3 need to be considered when driving. Boats and ships, unlike planes, also move across a surface (although it is 6 DoF), so they do not have full freedom of movement. As trains use rails controlling them is more straightforward. The field of flight related tasks for reinforcement learning has posed a big challenge due to its high complexity and continuous state space, especially prior to the introduction of DQN. That does not mean that no work was done in the field at the time, as is proven by Ng et al. [16]. However, DQNs made it much more manageable to train good, working policies and so since they were introduced, the work of RL in flight has increased significantly. There is a lot of work in many different areas of aviation to show that. There are works published in swarming [17], landing [18], heading control [19], navigation [20] and attitude control [12] among others. Even aviation industry giants like Airbus [21], [22], Lockheed Martin [23], [24] and Boeing [25], [26] have ventured into the area of RL research, further showcasing the active interest and validity of the field. One can divide the aviation related work in RL into two categories, the more popular category of research with quadcopter drones, and the less popular area of fixed-wing aircraft (planes). More work with RL is conducted using quadcopters [27], [28], [29], [30], [31], [32], [33], and therefore more attention is given to them. That, however, does not mean that fixed-wing planes have seen no attention at all. Fixed-wing aircraft come with a set of advantages and disadvantages when compared to quadrotor drones that differentiates the two fields and their use cases. While drones can start and land vertically and hover mid-air, fixed-wing aircraft are aerodynamically beneficial, can carry heavier and bigger loads and carry out longer endurance flights as well [32], which makes them suitable for a different set of circumstances. Fixed-wing aircraft are also used by airlines to transport roughly 700 million passengers annually [34]. This demands high

security assurance. As most crashes are linked to human error [35], [36] adding artificial intelligence (AI) and RL models to assist can improve the safety of the crew and passengers on board, therefore encouraging active work in the field. With that, research in the area of fixed-wing aviation has also picked up, in relation to where it was just a handful of years ago, it still remains rather dwarfed by multi-rotor drones. This often makes it rather difficult to find fixed-wing related papers, as both fields share and often use the same term UAV (Unmanned Aerial Vehicle), meaning that when searching for papers one often ends up having to go through numerous drone related papers until one can find one that fits their needs. In addition, fixed-wing reinforcement learning spans across multiple disciplines, and so do the papers, further complicating the search.

Papers included in this work were acquired in a few ways. Initially the search terms "reinforcement learning" AND "fixed-wing" and "reinforcement learning" AND "UAV" were used. The resulting papers were then screened manually to see if: they were actually using fixed-wing aircraft, as the term UAV also includes other types of aircraft and to see if the papers focused on control tasks. Later more search queries were run while adding the specific control tasks to the search (e.g. "attitude control"). Many papers were also found by looking through the references of papers that were discovered earlier in the process. A few papers were also discovered in simulator specific forums or repositories and a few other papers were discovered via their code-base first which then directed to the papers. Papers starting in 2015 were considered as this is when RL, deep RL and RL for UAVs have seen the biggest uptick in publications (see Figure 1).

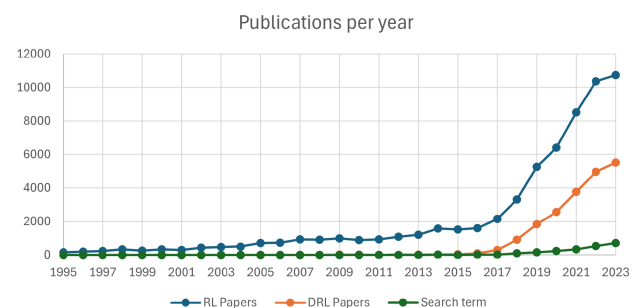


FIGURE 1. This graph depicts the number of papers published per year via the "Web of Science". The blue line indicates the number of papers received when searching for "Reinforcement Learning", the orange line the number of papers published when searching for "Deep Reinforcement Learning" and the green line when searching for the term: "reinforcement learning" AND ("UAV" OR "fixed-wing" OR "fixed wing" OR "aircraft" OR "aviation" OR "flying").

A. CONTRIBUTIONS

This paper will review the current state of RL for fixed-wing aircraft control tasks (attitude control, heading control, landing, flocking, taxiing, and more). To the best of our knowledge only a single review paper that focuses specifically on RL for Fixed-Wing aircraft was published to this day [37], with existing review papers almost exclusively

looking at drones or with fixed-wing aircraft only making up a small part of the review [27], [29], [33]. This is where this work will extend the current pool of available research papers. The previously mentioned review functions more like an introduction into the field and does not focus on providing much information on the works themselves. This paper hopes to go a bit more in depth and includes more papers, trying to explain their methods in more detail, while also giving an overview of the research currently available, in order to direct towards relevant works in the hope of helping researchers find papers that can help them advance their work. The authors have also decided to not include papers that are mainly focused on air-combat, dog-fights and other papers that focus mainly on military applications. For that we would recommend looking at the aforementioned review paper [37], as it focuses heavily on military use cases.

B. STRUCTURE

Section II gives a brief introduction into the basics of the fields of RL and fixed-wing control tasks. In Section III we introduce the available toolkits that have been created to facilitate RL research for fixed-wing aircraft. Next, in Section IV, the various different applications of fixed-wing RL are explored (Section IV-A: Attitude Control, Section IV-B: Heading Control, Section IV-C: Flocking, Section IV-D: Landing, Section IV-E: Taxiing and Section IV-F Others). In Section V we attempt to give an overlook at the current state of fixed-wing RL as well as giving an outlook into what could be next. Lastly, in Section VI we conclude this paper.

II. BACKGROUND

In current aviation related research with reinforcement learning, there are two large categories. One is the category of quadrotors, quadcopter or drones (and sometimes helicopters). This field is the one seeing more attention as of now [27], [28]. This paper however does not consider these quadrotor drones. There also is the branch of using non reinforcement learning ML or DL methods to interact with aircraft [38], [39], [40]. However, in this work we will look at fixed-wing aircraft (traditional airplanes). Fixed-wing aircraft are controlled via the actuation of three control surfaces, those being the ailerons, the elevators and the rudder (see Figure 5). The ailerons control the roll of the plane, the elevators control the pitch and the rudder controls the yaw. In addition to that, the throttle controls the thrust and therefore the airspeed and altitude.

A. CONTROL THEORY METHODS

Reinforcement learning, being a rather new technology, is new to aircraft control. Before RL was applied to these control tasks, other methods were commonly used. These control methods include: human pilot control, and control theory based methods. For motivation and context, we will describe a couple of the more well known control theory methods and then provide a table of other methods and related works.

1) CONTROL THEORY

Control theory [41], [42], [43], [44], [45] concerns itself with finding control methods that allow a system to reach a certain desired state starting from the current state it is in. To do that control theory methods try to find (an optimal) set of control inputs to the system to achieve the desired state. In terms of this paper, we can think of the airplane as the system to be controlled, the planes control mechanisms (e.g. control surfaces, throttle, landing gear, etc.) as the controls, and the effect these controls have on the plane and the way it transitions after application of said controls are the dynamics. Of course this only functions as an example and can vary from application to application, but this shows that an airplane fits the idea of control theory very well. One can also easily see the parallels between control theory and what RL offers. RL also controls (actions) a system (agent) that behaves dynamically (environment) to achieve a certain goal (via the reward function) through restraints (via punishments/negative rewards).

Some common control theory-based methods that have been used in aviation are: PID control and LQR.

2) PID CONTROL

Proportional-integral-derivative controller (PID) [46] is a control method that has received research attention for fixed-wing aircraft control [47], [48], [49] and is also often used as a benchmark in RL papers [13], [14], [50], [51]. PID calculates an error value. This error value represents the difference between a targeted value (state) and a measured value (state). To alleviate this error value, a correction value is computed. This computed value depends on 3 parts known as the proportional term, the derivative term, and the integral term. The proportional term calculates control actuation proportionally. This means that the closer the agent (e.g. aircraft) is to the desired state, the weaker the control actuation and vice versa. The integrator is added to the output of the proportional term and sums up the integrated error over time (as a running total) and can therefore use its memory of past information to improve the control signal by removing constant errors. The derivative term is the last term and also added to the two other terms. It is in charge of taking the future into consideration. It does that by using the rate at which the error is changing (e.g. If the error is decreasing fast, then lower the control signal strength more. If it is only slowly decreasing, then the control value is also only slightly decreased). Each of these three terms can be weighted differently depending on the application. PID is calculated via:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (1)$$

where $u(t)$ represents the control signal at time t . The term $e(t)$ represents the error at time t (the difference between the desired and actual values). The values K_p , K_i , K_d are the proportional, integral, and derivative terms, respectively. The first term is the proportional term, the

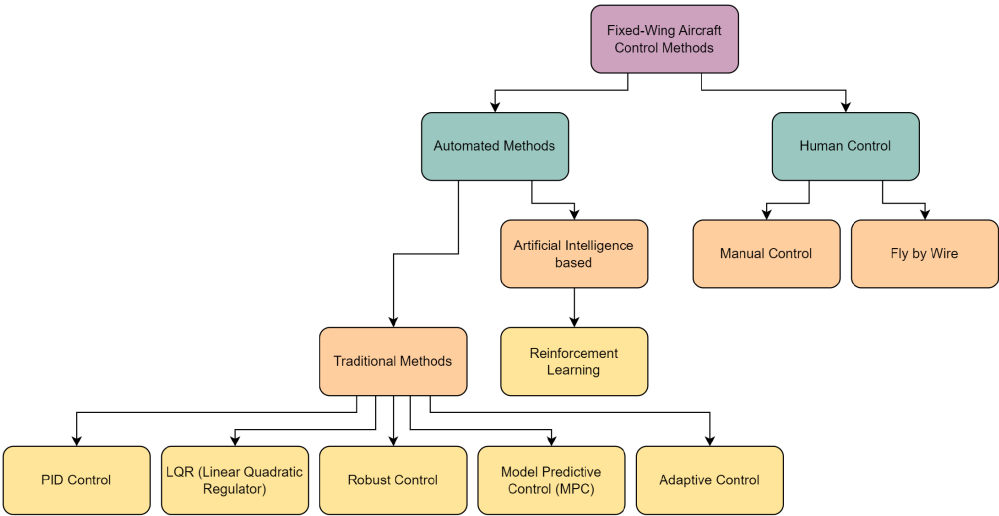


FIGURE 2. This mind-map illustrates different control methods for fixed-wing aircraft.

second term is the integral term, and the third term is the derivative term mentioned above. PID also finds use in automotive systems, robotics and other areas.

3) LQR & LQG CONTROL

Linear quadratic regulator (LQR) and linear quadratic Gaussian (LQG) control [52] are control methods that have both been applied to aircraft control before. The LQR methodology provides optimal controller settings by optimizing a cost function of a set number of variables (states), where each variable will be assigned a cost which will then be minimized. At the same time, the control actions are also considered and also minimized in terms of their cost. For example, the deviation of the current state of the aircraft could be considered via a state matrix, where each cost value for each state variable has its own weight associated (e.g. the pitch angle could be weighted more than yaw) with it. Actions are considered in an action term, where actions are also associated a weight that represents the cost of the action (e.g. using more throttle uses more fuel and is therefore considered expensive). This approach clearly tries to find an optimal solution that minimizes the cost in state deviation while also minimizing the cost of actuation. So say we have a system with two important goals. These two goals could be the delay in actuation, and the cost in actuation such as energy use, etc. More formal definitions of the LQR problem can be found in [53], [54], [55]. The LQG control method is very similar to LQR but adds additional parameters to deal with noise.

4) OTHERS

Other control methods that exist but are less common and less used are included in Table 1. They will not be discussed in detail in this paper as they are rather uncommon, but references are provided to papers that implement them for

aircraft control as well as general references that explain each approach in detail.

TABLE 1. Other less common control methods that have seen research for aircraft control.

Control Method	Relevant References
Robust Control	[56]–[59]
Model Predictive Control	[60]–[62]
Adaptive Control	[63]–[65]

B. REINFORCEMENT LEARNING

Reinforcement learning is a part of machine learning (ML) and deep learning (DL) [66], [67]. It does however work quite differently than supervised learning and unsupervised learning do (see Figure 3). Where both of the aforementioned methodologies rely on a existing dataset to train on, meaning that both these methods need and use a dataset that is present even before the training starts. In reinforcement learning, there is no need for a dataset before training starts. RL can operate in a markov decision process (MDP) framework and typically runs on a live environment of which the agent has no prior knowledge. RL receives data from the environment at runtime and also interacts with it at runtime through so called actions. These actions are picked from a pre-defined and task relevant list of possible actions, and each action affects the environment in a way. The agent starts off picking actions at absolute random and learns to pick the best possible actions over time. This agent environment interaction sets the agent up to learn how to successfully achieve tasks in the environment through trial-and-error learning [66]. This means that the agent is at no time given any direct instructions on what it should do (as would be the case with supervised learning), but rather the agent receives a reward value and a state representation each time it applies an action to the

environment. The state is a numerical representation of what is happening in the environment and can be image data [2] or numerical data [12]. This state is the only way the agent “sees” what is going on in the environment, and needs to be picked carefully because of that. It needs to do a good job of portraying all the task important aspects of the environment to the agent, preferably in a data-efficient manner. The reward value represents how well the agent is performing at this instance. In Q-learning, for instance, the Bellman Equation (see (2)) is used to update the value representation of the current state. Due to this, the agent will learn over time which action is best at what time, all on its own. This shows why it is so important to formulate a good reward function for the task at hand, since without one the agent will have a near impossible challenge ahead of it.

$$Q(s_t, a_t) = Q(s_t, a_t) + lr * (r + \gamma * Q(s_{t+1}, a_{t+1})) \quad (2)$$

In (2) Q stands for the Q-value for state s_t and action a_t at time t . lr is the learning rate, r the reward, and γ is the discount factor. As such it uses the old state-action pair Q-value $Q(s_t, a_t)$ and adds the current reward r (at timestep $t + 1$ for state s_{t+1} when using action a_{t+1}) it just received plus the expected future reward at the next state (the highest Q-value at the next state s_{t+1}). It is multiplied by the learning rate that impacts how fast it changes the current Q-value and the discount factor limits the amount that future rewards contribute to the Q-value.

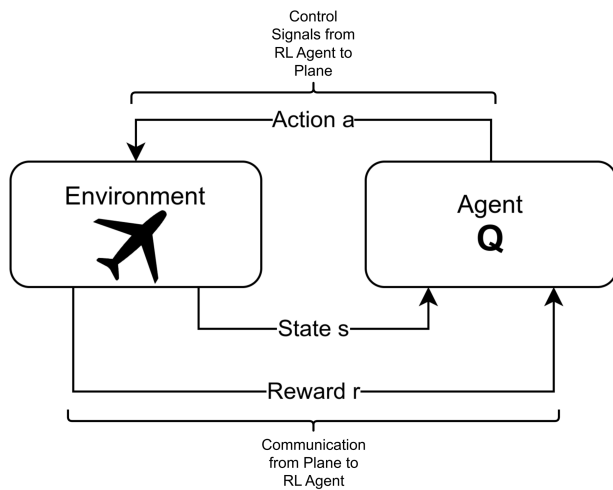


FIGURE 3. A chart displaying the workflow of value based RL training. The agent sends control signals based on the state to the environment and then in turn receives the new state and the reward of the just performed action.

1) VALUE BASED REINFORCEMENT LEARNING

In value based reinforcement learning, the algorithm tries to predict certain values for states or state action pairs. These values predict the expected future reward of the state or state action pair. This tells the agent, that the best possible action to take at the current state is the one that has the highest value attributed to it. The policy of a fully trained agent is therefore made up from picking the highest value at every step. During

training, however, randomness is introduced, which enables so called exploration. Due to exploration, many actions at the states will be tested out by the algorithm to learn which ones work best. Common and popular NN and value based hybrid approaches include Q-learning [3], deep Q-learning [1], [2] and double deep Q-learning (DDQN) [68], [69], just to name a few.

The link between Value (Q^*) and Policy (π^*) can be very subtle. But having an optimal Value function can lead to having an optimal Policy. In its simplest form, this relationship can be seen in the following formulation.

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} Q^*(s, a) \quad (3)$$

2) POLICY BASED REINFORCEMENT LEARNING

In policy based approaches, the agent will learn the policy directly. Instead of it learning Q-values alone, the agent learns parameters (e.g. weights, probabilities) similar to the ones used in classification problems used in deep learning. One advantage of advanced policy methods is that the actions can be continuous, something that value based approaches can not really do. Policy based methods also tend to work well in highly complex action-spaces and they tend to learn more smoothly than their value based counterparts.

3) RL ALGORITHMS

There are many different RL algorithms available and all of them have their differences. Some are inherently better suited to aircraft control than others, but with the right modifications or adjustments, one can adapt these algorithms to work for the task at hand. It becomes clear, when looking at Figure 4 that PPO and DDPG have so far been the most commonly used algorithms. Algorithms such as PPO and DDPG, among others, have gained a lot of attention due to their performance in other areas and have therefore also made their way into fixed-wing RL. Not only popular pre-existing algorithms have seen much use. Some researchers have modified less popular algorithms to fit their needs. We will explain the more commonly used algorithms here, as well as DDQN and DQN.

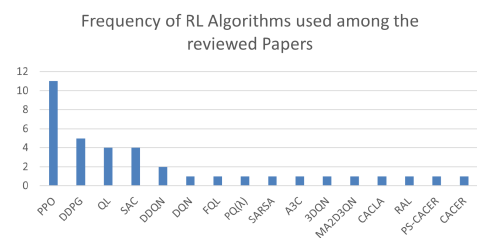


FIGURE 4. The frequency of RL algorithms used in Tables 5.

a: Q-LEARNING

Q-learning (QL) [3] is a traditional RL algorithm that stores Q-values (Q-values represent the maximum expected future reward) for state action pairs in a lookup table that is called Q-table. At each time-step the agent will look at all possible actions for said state and pick the action with the highest

Q-value (when exploiting). Said action will then be applied to the environment and using the reward it received the value for the chosen action at that time-step will be updated via the Bellman equation (2). When exploring, the agent will pick a random action, ignoring whether or not it has the highest Q-value currently. The Bellman equation will still be applied to the state-action pair that was randomly chosen to update its values. Like this, over time, the agent will find a near optimal control strategy to solve the task given to it via the environment in a way in which it maximizes the rewards. The big downside is that QL can not generalize to new states and difficult tasks would therefore need to explore nearly endless amounts of possible state-action pairs, which eventually becomes infeasible.

b: DEEP Q-NETWORKS

Deep Q-network (DQN), as introduced by Mnih et al. [2], is the RL algorithm that is arguably responsible for the boom RL has experienced since its publication in 2015 (see Figure 1). DQN combines deep neural networks with RL (where the neural network will replace the Q-table and function as Q-value function approximator) while introducing a few mechanisms that allow the algorithm to outperform older approaches while being applicable to many different use-cases. DQNs still utilize the Bellman equation (see (2)) to set the labels for the regression task of the neural network. DQN uses memory replay, a list of n previous states, the action taken, the next state and the reward received. A subset of a certain batch size will be drawn randomly from said replay-memory to be used for training at each training iteration to help with training. Without this, only a single data point would be accessible for training, which can lead the algorithm to overcompensate for said single step. DQNs also feature two neural networks (the target network and the evaluation network). They both have the same architecture and only one (the evaluation network) is used for training, the other network (the target network), which is used for predictions, will copy the weights of the evaluation network every few epochs. This also makes the DQN more stable and prevents overcompensation to small changes. All this allows DQN to take in high dimensional and continuous inputs (but still only discrete actions) while being able to generalize to before unseen states (something that Q-learning can not do), allowing DQN to function in ever more complex environments.

c: DOUBLE DEEP Q-NETWORKS

Double deep Q-networks (DDQN) [69] are very similar to DQN as they are built on top of that concept in order to provide more stability and robustness while minimizing the overestimation bias that DQN tends to have. To do so, the way the two neural networks that make up DQN are utilized is slightly modified in DDQN. Instead of having the target network do the entire prediction process alone, now the evaluation network predicts which action to take, but the target network will then predict the Q-Value for the action

picked by the evaluation network, thus reducing the impact of the before mentioned overestimation bias and in turn training stronger models. Where in DQN the prediction of the action and its Q-value would look like (4)

$$Q_{DQN} = r + \gamma * Q(s, \operatorname{argmax}_a(a; \psi_T); \psi_T) \quad (4)$$

it will look like (5) in DDQN. The greek letter ψ symbolizes the parameters of the neural networks in both equations, r is the reward, γ the discount factor and s is the state. E denotes the evaluation network in these equations while T denotes the target network. Here we can see how both networks are now doing the predictions together.

$$Q_{DDQN} = r + \gamma * Q(s, \operatorname{argmax}_a(a; \psi_E); \psi_T) \quad (5)$$

d: DEEP DETERMINISTIC POLICY GRADIENT

Deep deterministic policy gradient (DDPG) [70] can be thought of as a DQN that can handle continuous action spaces [71], because DDPG uses similar techniques such as the replay buffer (which adds stability and diversity to the training) and target networks (to keep the agent from drifting too much). But, unlike DQN, DDPG is an actor-critic method, meaning that it uses a deterministic policy function (it learns the policy directly) to pick the greedy action for a state. It does so by using four neural networks (not only two like DQN) to learn, those being the critic network (the Q-value network), the actor network (the policy network) and, just like in DQN, a target network copy of both. The actor network predicts continuous action values via the state as input and the critic net is a value approximator and predicts Q-values, with states and the action values of the actor as input. The Q function critic is a neural network learned by parameters ϕ , and the policy actor model is a neural network learned by parameters θ .

More formally the DDPG algorithm workflow is as follows. First, we initialize two neural networks (policy and the Q function). We additionally define a replay buffer that will hold previous transitions to sample from. Transitions represent state, action pairs and results provided back by the environment. We also create copies of the policy and Q function models (the target networks). These copies are time delayed and serve to keep the model from drifting too much. Iteratively, we select an action given the state. Taking the action in the state returns the new transition information. This transition is stored in the replay buffer. Now the neural networks are updated via gradient descent. To do this, a batch of transitions is sampled from the replay buffer. This batch of transitions is used to calculate targets which serve to update the Q function.

$$y(r, s', d) = r + \gamma Q_{\phi_{\text{target}}}(s', \pi_{\theta_{\text{target}}}(s')) \quad (6)$$

Notice that in (6) we use the target networks to predict the target Q-values. With the calculated targets the Q-value function can be updated via the following loss.

$$L(\phi) = \frac{1}{\text{batch}} \sum_{\text{batch}} (Q_{\phi}(s, a) - y(r, s', d))^2 \quad (7)$$

We can then maximize the actor model with the policy loss, as seen in (7).

$$L(\theta) = \max_{\theta} \hat{E}[Q_{\phi}(s, \pi_{\theta}(s))] \quad (8)$$

The final step in the epoch is to update the target networks via (8).

e: SAC

Soft actor-critic (SAC) [72] is a stochastic algorithm and introduces an entropy term. This entropy term encourages, and therefore helps with, exploration. The SAC method is very similar to the DDPG method that was previously described. Another key difference to DDPG is the fact the SAC is a stochastic approach. This means that the actor network does not simply output a deterministic action for a state that is deemed the best, but rather returns a probability distribution (Gaussian mean and log standard deviation) over all actions. This helps with exploration as it allows for added uncertainty during training, through which the agent will be exposed to more diverse policies. The entropy term mentioned above is introduced as:

$$-[\alpha \log \pi_{\theta}(a|s)] \quad (9)$$

With this entropy term, the policy gets an additional reward signal at each step of training which is related to the current entropy. As can be seen below, the SAC equations are similar to the DDPG equations but including the entropy term.

$$L(\theta) = \mathbb{E}_a [Q_{\phi}^{\pi_{\theta}}(s, a) - \alpha \log \pi_{\theta}(a|s)] \quad (10)$$

$$\begin{cases} y(r, s', d) = r + \gamma (Q_{\phi}^{\pi_{\theta}}(s', \hat{a}') - \alpha \log \pi_{\theta}(\hat{a}'|s')), \\ \hat{a}' \sim \pi_{\theta}(\cdot|s') \\ L(\phi) = \mathbb{E}_{s,s'} [(Q_{\phi}(s, a) - y(r, s', d))^2] \end{cases} \quad (11)$$

Here too the loss in (10) and in (11) will be optimized. The parameter α is controlling the trade-off between exploration and exploitation. R is the reward, s the current state, s' the next state, π the policy and a the executed action [37].

f: PROXIMAL POLICY OPTIMIZATION

Proximal policy optimization (PPO) [73] is a policy gradient method used in RL which has been very successful. The goal of PPO is to update the network policy efficiently. The network policy is updated via a step size using gradient learning. One of the goals with PPO is to control this step size. As such, PPO adds clipping to its objective function, which disallows it from making updates to the network that are deemed to be too big or too small, allowing for smooth training and convergence.

$$\begin{cases} J = \hat{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)] \\ r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} \end{cases} \quad (12)$$

The actor has to maximize (12) in order to learn. Here a represents the action, s the state, t is the current time step, π is the policy network used to make decisions, θ are the parameters of the policy, and $\hat{A}_t$ is the advantage function [37]. The advantage function is really important and helps to determine the relative value of the current action via the feedback from the reward model. The term $r_t(\theta)$ is a probability ratio created to capture the difference between the old and new policy based on the update. This ratio allows the control of the difference between old and new policy networks using clipping. The goal is to keep the policy from changing too much in one update, and therefore controlling divergence. Finally, the minimum of the clipped versus un-clipped ratios is selected.

4) ADVANCES

With these ever improving new algorithms, that allow for multi-dimensional and highly complex input and for continuous outputs within those highly complex environments, ever more difficult tasks are being attempted to be solved via RL. Whereas before, the limitations of tabular Q-learning or the limitations of DQNs only allowed for discrete actions, newer algorithms now offer solutions to these limitations. As mentioned above, these new approaches improve the learning, eliminate problems such as over-estimations, offer smoother convergence, and more.

Today, aircraft can count on many sources of information whether image based or sensor based. With this information, complex tasks such as multi aircraft flocking, 6 degree of freedom attitude control, accurate landing, among many other problems, have now proven to be feasible. With many works now moving from such feasibility studies to research that tries to improve upon these findings, be it to make the process more efficient, to train better performing agents or to explore different approaches to the problem.

5) CHALLENGES, LIMITATIONS & OUTLOOK

RL in regards to fixed-wing flight still has a long way to go. As of right now there are essentially no standard metrics with which researchers can compare their results, researchers most often opt to develop their own toolkit based on various simulators or physics models, use different aircraft, different state representations, action spaces, algorithms, metrics and so on. This makes it hard to directly compare two studies to one another. Other fields have standard datasets, methodologies and metrics, which fixed-wing RL still lacks. Researchers would benefit if such a standardized approach could be developed.

a: REINFORCEMENT LEARNING WITH HUMAN FEEDBACK

Approaches such as RL with human feedback offer interesting perspectives. RL with human feedback (RLHF) is a very promising field, which will, for example, allow for a deep neural network to approximate a suitable reward function, without researchers having to formulate one by

hand [74], [75], [76], [77]. This falls in line with the idea of shifting towards data-driven ML and DL [78]. Therefore, in traditional RLHF we need to train two neural networks. One for the RL policy and one for the reward model. The reward model is usually trained via human feedback. RLHF tries to find the optimal reward model for use in training. As such, two networks are needed. Simplistically, a reward model can be obtained from training a binary classifier. Given a state, the policy selects two actions. These two sets of state/action pairs are labeled by a human (e.g. as good or bad) and this labeled data is used to train a binary classifier (the reward model). Once trained, this reward network can be used to optimize the policy network.

As part of the outlook, we present and discuss direct preference optimization (DPO). DPO presents a recent breakthrough in RL research that may have a huge impact in aircraft automation.

b: DPO

Direct preference optimization (DPO) [79] is a groundbreaking methodology that can have a huge impact on RLHF. Previous approaches have required a policy model and a reward model. DPO removes the need for a reward model, therefore, simplifying the whole process. In their paper, the authors showed that DPO can fine tune models using a simple classification loss. Through mathematical intuition, the reward model is incorporated into the loss function used to optimize the policy network. As previously stated, in RLHF we need a policy network and a reward network. The policy network can be fine tuned via human preferences. The policy network should learn to produce results similar to the human actions (e.g. the pilot). Here we can assume that given states, the model can predict two actions to take. These results are presented to humans for review. The labelers indicate which produced action is better (denoted as $y_w > y_l$).

The human preferences can be learned by a reward model and the corresponding distribution can be formulated as follows. The Bradley-Terry (BT) model stipulates a formulation for the human preferences distribution as follows:

$$p^*(y_1 > y_2|x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))} \quad (13)$$

Based on the previous formulation, the parameters of the reward model can be learned via maximum likelihood. The objective can be expressed as a binary classification loss. This loss can be expressed as follows.

$$J = \hat{E}[\log_\sigma(r_\theta(x, y_w) - r_\theta(x, y_l))] \quad (14)$$

Once the reward model is trained, we can use it in the next RL phase to train the policy network via the following objective:

$$\max \hat{E}[r_\theta(x, y) - \beta_{KL}[\pi_\theta(y|x) || \pi_{ref}(y|x)]] \quad (15)$$

The main innovation is that DPO leverages a choice in the reward model “parameterization” that simplifies the

problem. Their key insight is to use “analytical mapping” from the reward function to the policy.

This allows for the integration of the reward loss into the policy loss. As such, the policy model implicitly includes the reward model.

In the DPO paper, the authors showed that the previous KL-constrained reward maximization objective can be expressed (in its optimal solution form) as follows:

$$\pi_r(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right) \quad (16)$$

The $Z(x)$ term is called the partition function and must also be calculated. As such, this solution of the objective function is not practical to use. However, this is where the analytical insight came that created the DPO method. The previous formula can be rewritten as follows where the reward function can now be computed entirely from the policy and the $Z(x)$.

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x) \quad (17)$$

With this insight and several conditions discussed in the original paper, it was proven that the reward model used in the previously described Bradley-Terry (BT) model [80] can be expressed in terms of the previous equality. Additionally, the intractable term $Z(x)$ cancels out. After further simplification, the Bradley-Terry (BT) model can now be expressed entirely in terms of the policy and a reward model is no longer needed.

$$p^*(y_1 > y_2|x) = \frac{1}{1 + \exp(\beta \log \frac{\pi^*(y_2|x)}{\pi_{ref}(y_2|x)} - \beta \log \frac{\pi^*(y_1|x)}{\pi_{ref}(y_1|x)})} \quad (18)$$

And from here, a new loss (below) can be formulated entirely in terms of policies without the need for the reward model. This means the policy network can be trained without the need of a separate reward model.

$$J = \hat{E}[\log_\sigma(\beta \log \frac{\pi^\theta(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi^\theta(y_l|x)}{\pi_{ref}(y_l|x)})] \quad (19)$$

Possibly removing reward functions from RL is a large leap and can in the future have a high impact in RL based aircraft automation.

III. RL FIXED-WING FLIGHT SIMULATION TOOLKITS

When one wants to run reinforcement learning experiments using fixed-wing aircraft, a methodology has to be decided upon to conduct the experiments with. There are two major categories to choose from. One could either choose to pick a physical real life airplane to conduct the experiments with. While this is definitely possible, this is, for the most part, very time consuming and potentially also very expensive. The reasons for that are twofold. Reinforcement learning is usually entirely based upon a trial-and-error learning approach, which means that agents start off with absolutely no knowledge of the task that they are meant to solve. Agents essentially act at random in the beginning of the training process. If that is done with physical planes, then

the plane is going to crash, or at least misbehave, a lot during early episodes, which could become pricey very fast. As for the time factor: reinforcement learning is a long process. Training happens through thousands if not millions of training episodes. If a real physical plane is used, then there is of course no way to speed up time, and the training would take a very long time to conclude. This does not mean that training with physical models is impossible, but it is less convenient. An arguably more practical approach would be to conduct training in software. If one wants to use real-life models for RL, then it would make most sense to use them during testing, to see whether the trained agent can transfer its policy from simulation to real planes and to test its robustness. Since flight-dynamic-models and flight simulators have come a long way and have accomplished near perfect flight physics simulation, using said models and simulators for reinforcement learning tasks becomes a valid, cheap, accurate, fast and convenient alternative. Simulators allow for no cost training, where no costs arise if a plane does indeed crash, a faster than real-time simulation speed, meaning that experiments are able to be conducted much faster, and they do all that with physics that are highly accurate. There is a selection of toolkits made to aid researchers in the process of conducting such experiments. These toolkits offer researchers with a code-base that is already tailored towards RL and make it very easy to set up and run RL experiments with fixed-wing aircraft, without having to start at nothing.

A. AVAILABLE TOOLKITS

There is a fair amount of toolkits available. Each one of them is a bit different, but they all serve the same overall purpose. They are all designed as to aid researchers in creating and running RL experiments with fixed-wing aircraft. A list of the toolkits that were found in the preparation of this manuscript can be found in this section as well as in Table 2.

1) GOR-REN/GYM-JSBSIM

Rennie's Gym-Jsbsim toolkit [86], which was developed as part of his master's dissertation [19] is built on top of the widely popular OpenAI Gym [87] workflow, which means it is compatible with baselines [88] and stable baselines [89], [90], meaning it supports a wide array of RL algorithms, without them having to be implemented by hand. It uses the JSBSim simulator [91], which is very well suited for RL tasks, due to it being able to run headless and at faster than real-time speed. The toolkit features heading control scenarios by default.

2) GYM-XPLANE

Lukmans GYM-XPLANE [81] toolkit supports heading control out the box, while using the X-Plane [92] simulator. X-Plane is a commercially marketed flight simulator that boasts very detailed graphics and highly accurate physics. However, X-Plane does not support headless mode or sped

up simulations. The GYM-XPLANE toolkit also follows the Gym architecture, therefore allowing baselines to function in combination with it.

3) GYM-FIXED-WING

Bøhns toolkit [13], [93] runs on the PyFly simulator [94], which was also developed by Bøhns as part of his fixed-wing RL research. It does not support 3D rendering, but does plot the flight dynamics to graphs for visualization. The toolkit too follows the Gym approach.

4) GALLEON/GYM-JSBSIM

Alleons Gym-JSBSim [82] toolkit is a forked version of the Rennie toolkit introduced earlier in this section. This version of the toolkit extends extensively on the version as it was released by Rennie. It keeps the basic idea in place, running with JSBSim and Gym, but introducing new features like taxiing or altitude control tasks.

5) MARKOV-PILOT

Ecksteins Markov-Pilot [83], [95] is another fork of the toolkit published by Rennie, and was published as part of Ecksteins Thesis. It again features many of the same characteristics like JSBSim and Gym support and also extends the original version with new features and new control tasks.

6) QPLANE

QPlane [84], [96] is a toolkit that has not yet incorporated Gym support, but rather features its own approach. That said, currently, without Gym support, the number of RL algorithms in QPlane is limited since it does not support baselines. QPlane also differentiates itself from the other toolkits by the fact that it does not just include one simulator, but two that work interchangeably (Those being XPlane 11 [92] and JSBSim [91], [97] with FlightGear [98] rendering), which allows for more possible train-test setups and scenarios. It comes with attitude control as its default task.

7) FIXEDWING-AIRSIM

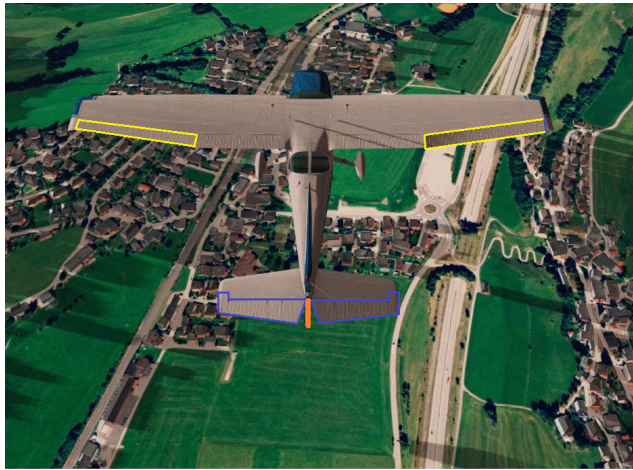
The Fixedwing-Airsim toolkit by Quessy combines Microsoft's Airsim [99] with the open source simulator JSBSim. It also follows the Gym methodology and therefore works with baselines too. It comes with landing and flight tasks.

IV. FIXED-WING AIRCRAFT RL CONTROL TASK EXPERIMENTS AND FINDINGS

Fixed-wing aircraft allow for a multitude of different tasks that reinforcement learning agents could try to solve. One could even say that mostly all the tasks that a fixed-wing airplane performs could be attempted to be solved via reinforcement learning. Currently, a number of fixed-wing aircraft control tasks have already been attempted and solved to a certain extent using reinforcement learning methods. While some are more obvious, like "navigation" and landing

TABLE 2. Toolkits available that facilitate Fixed-Wing RL.

Ref	Author	Toolkit	Simulator	Base	Algorithms	Tasks
[19]	Rennie et al.	Gym-Jsbsim	JSBSim	Gym	Baselines	Heading
[81]	Lukman et al.	Gym X-Plane	X-Plane	Gym	Baselines	Heading
[13]	Bøhn et al.	gym-fixed-wing	PyFly	Gym	Baselines	Attitude
[82]	Alleon et al.	GYM-JSBSim	JSBSim	Gym	Baselines	Heading, Altitude, Taxi
[83]	Eckstein et al.	Markov-Pilot	JSBSim	Gym	Baselines	Control
[84]	Richter et al.	QPlane	X-Plane, JSBSim	Own	QL,DQN,DDQN	Attitude
[85]	Quessy et al.	Fixedwing-Airsim	JSBSim	Gym	Baselines	Landing

**FIGURE 5.** Image depicting the usual control surfaces of a Fixed-Wing Aircraft, taken in GeoFS [100]. The ailerons, that control the roll, can be seen in yellow, the elevators that are responsible for the pitch in blue and the rudder that handles yaw can be seen in orange.**FIGURE 6.** This image shows the 3D coordinate frames of an airplane. Yellow shows the roll axis, blue depicts the pitch axis and orange the yaw axis. Screenshot taken in GeoFS [100].

tasks, others might not come to mind instantly when one thinks about the tasks that an airplane has to complete to successfully complete flight. Tasks like inner loop attitude control for example are just as important as landing the airplane, so researchers have tried to solve this problem with RL too. In this section, different control tasks will be explored, explained and then looked at from the angle of RL, looking at what work has already been done in the field and also summarize applicable works in Tables 5 & 6.

A. ATTITUDE CONTROL

Attitude control is the act of inner loop control of an aerial vehicle. This includes the control of the pitch and roll of the plane. When talking about attitude control of fixed-wing aircraft, then the aileron and elevator (see Figure 5) control surfaces are used to control roll (see Figure 7) and pitch (see Figure 8) of the plane respectively. One could either design a system that trains agents to maintain a certain attitude, or one that moves towards a certain attitude from a different starting orientation. One could also differ between a system that tries to reach perfect level attitude (always the same, e.g. 0° , 0° for pitch and roll) and a system that is able to reach any attitude that it is given, allowing it to not just stabilize towards one constant orientation, but to any possible orientation. Lastly, agents can be limited to only one of the degrees of freedom (DoF), say only pitch or only roll.

1) CASE STUDY

In 2019, after RL was in the midst of the boom that Mnih et al. and Silver et al. had caused, fixed-wing aircraft attitude control also regained popularity and again started to see attention. Bøhns et al. [13], [101] use the proximal policy optimization (PPO) [73] RL algorithm. The agent has control over the elevator, ailerons (with the aileron and elevator controls being mapped to the elevon due to the nature of the plane used in this study) and the throttle as the action space. Due to the fact that Bøhns et al. opted for PPO, they were able to have the agent produce fully continuous action values and apply these to the agent directly via the PPO algorithm. This paper uses a state space consisting of the roll and pitch angles, the current angular velocities, the current airspeed and the state errors. The paper also used the last 5 steps of these values as inputs as well to further enable learning of the dynamics. Lastly moving averages of the previous actuator setpoints were also added. It was also noted that this smaller state space outperforms more complex ones that they experimented with. The reward was calculated via the distance between the current attitude and the desired one (roll, pitch, airspeed) and a cost for changing actuator setpoints. It was noted that the reward function was not a major focus of this work. The frequency at which the agent was used to predict new commands was every 0.1 seconds and it falls in line with the standard 100Hz frequency of many auto piloting systems. Training episodes were conducted for 20 seconds of flight time, corresponding to 2000 timesteps per episode. The agent was then compared to a PID controller in the same environment and across the same set of metrics



FIGURE 7. Image depicting an airplane from a heads on perspective, it displays how roll will affect the airplane's attitude (leaning left/right). Screenshot taken in GeoFS [100].



FIGURE 8. Image depicting an airplane from a side-view, showcasing how pitch affects an aircraft's attitude (nose down/nose up). Screenshot taken in GeoFS [100].

and it was found that the RL agent managed to achieve similar results to PID while also stating that the agent generalized well to scenarios never before seen during training. In 2023, Bøhns et al. [102] extended on said work and implemented a data efficient approach (utilizing findings from [103], [104] and [105] where the model (using soft actor critic (SAC) [72] RL) can learn directly on the original nonlinear dynamics and does so in very minimal time. The state space was enlarged to include more information and the rewards were switched to a sparse function. The final model is then tested on a physical plane in a field test, the first of its kind according to the authors, showing good results that are comparable to the ArduPlane proportional-integral-derivative (PID) controller.

2) PITCH ONLY

In pitch only attitude control the agent is only responsible for a singular axis, that being the pitch axis. The RL agent is generally tasked with reaching a target pitch angle, usually by taking control over the elevators of the plane through control commands generated by the RL algorithm. Better rewards are usually given the closer the agent is to the target pitch angle and the state usually encompasses the current attitude of the plane, focusing on the pitch DOF.

In 1997 Monaco et al. [106] conducted what might be the first experiments with RL for fixed-wing aircraft. The authors conducted these experiments with high-agility aircraft that were tasked with recovering from a non ideal pitch state. The agent was trained using residual advantage learning (RAL) [107], [108] and was given 5 actions, all of those corresponding to different values for pitch control. Later on another experiment was run where the action space was increased to 9 possible actions. The state space consisted of approximately 30 different values (and later 180) and the rewards were given at each time-step. It was reported that the results were promising, however not yet fully satisfactory in their current state at the time. Most early work of RL in flight was however conducted with helicopters [16], [16], [109], [110], [111] and will therefore not be discussed in more detail. The field of RL in fixed-wing aircraft only recently gained noticeable interest with the re-emergence of

RL as a whole after the work by Mnih et al. [1], [2]. With them improving RL's robustness and generalization ability and showcasing the new capabilities of RL, researchers have also started applying RL to fixed-wing aircraft again. But again, fixed-wing aircraft were not the main focus of aviation RL. Nowadays, with the newfound popularity of quadcopter drones within the last few years, they have become the main point of interest in aircraft RL [12], [112], [113], [114]. Nevertheless, fixed-wing aircraft do also see attention and are continuously improved, using different toolkits (see Section III) and various new RL algorithms.

Huang et al. [115] also present a attitude control agent for fixed-wing aircraft, but here double deep Q-learning (DDQN) [68], [69] is used. Here the state space is kept really simple, consisting only of the pitch angle deviation and pitch angular velocity deviation. The action space on the other hand is rather large, as it consists of 40 different variations of pitch control values (ranging from -10 to 10 in 0.5 steps). As can be seen only the pitch is controlled in this paper. The reward is calculated via the previous actions and the difference between the current state and the desired state. The agent is said to have the ability to stabilize to a good level.

Wada et al. [116] published a paper in which they control 1 Degree of Freedom (DOF), that being pitch, in a wind tunnel setting, meaning on a physical model of an airplane using an asynchronous advantage actor-critic (A3C) [117] implementation. The state space was of size 5 and was made up of the error between desired and current angles of attack, the angle of attack, pitch rate, elevator angle and the wind speed. The actions, which were discrete, varied, as the authors tried 3 different action space setups, all of which were containing different combinations of elevator angle rate values. The 3 action spaces were $[-10, 0, 10]$, $[-30, -10, 0, 10, 30]$ and $[-50, -30, -10, 0, 10, 30, 50]$. The reward was shaped to always be positive via a constant and was otherwise made up of the angle of attack error and an action penalty

that also varied between the 3 options. It was shown that the controller with larger action rates did best.

In [118] Zahmatkesh et al. apply pitch only attitude control to truss-braced wing aircraft (which are known to be less stable). They do so using an improved Q-learning methodology with fuzzy action assignment (FAA), which allows to generate continuous action values. The state space is the pitch angle and the pitch angular velocity, the action is the control of the planes pitch axis. The reward function takes into consideration the amount at which the elevator is actuated, as well as the delta between current and desired state.

Chowdhury et al. implemented an agent for longitudinal flight control in [119] (pitch only). This agent is trained in a simulation environment, but will be tested on a real life physical model airplane. The agent was trained via PPO in this work. The state is comprised of the airspeed, roll, pitch, yaw, angular velocities, longitudinal velocities, accelerations on (x,y,z), as well as reference values for airspeed, roll and pitch. The actions allow the agent to manage throttle and elevators respectively. The reward function considers the following values: airspeed angle error, pitch angle error, non-zero pitch rate, pitch rate, elevator control rate and throttle control rate.

3) MULTI DOF

In multi DOF attitude control the pitch is no longer the only axis of relevance and the plane most often also has to consider roll during the control task. As such this makes this task more challenging. The RL agent is often given control over the elevator and ailerons of the aircraft to move the plane into a desired attitude. States again mainly focus on portraying the current attitude (often via the planes rotation and angular velocities) and rewards are most often given based on the deviation from the current state to the desired one.

Zhen et al. [14] uses PPO in their work and interfaces with ROS [120] and Gazebo [121] for flight simulation. The state features include the planes rotation (roll, pitch, yaw), the angular velocities as well as the airspeed, the action space are the control surfaces and the reward function is made up of the current-desired state delta as well as the airspeed. The agent manages to reach PID level control.

Richter et al. used basic Q-learning (QL) [3] to try and solve the attitude control problem in [122] in an effort to see whether or not basic Q-learning would be capable of handling such a task. The state-space was therefore binned and encoded (pitch and roll) to a final size of 168 and the action space was 4, with the values corresponding to ailerons $+/-$ and elevators $+/-$, with the magnitude of the action based on the delta between the desired and current state on that axis. The reward was a continuous reward that considered roll and pitch angles. In [123] and [124] the previous work was extended by replacing the Q-learning algorithm with the DDQN algorithm and in turn also changing the state observation to a vector that was made up of deviation of the current pitch angle deviation, the roll axis deviation, the pitch rate, the roll rate, the yaw rate, the angle of attack and the

angle of sideslip. Both approaches have shown promising results and show that the methods are capable of learning and controlling the plane.

Li et al. [125] present an approach using the deep deterministic policy gradient (DDPG) [70] algorithm and L1 adaptive control algorithm [126] is added on top to improve robustness in uncertainty. The reward is shaped in a way in which it considers overshoot, constraints as well as the delta between the current and desired state. The actions that are being controlled are pitch and roll.

Qiu et al. [127] propose attitude control for a moving mass-actuated UAV using DDPG. The action space consists of the offset of the slider and the elevator and rudder deflection angles. The state space contains the error of the attitude at the current and previous time-step.

In [128], Chowdhury et al. extend their previous work by introducing lateral control to the agent (roll). The agent still uses PPO. The state space has remained the same, but the action space was extended to now also include ailerons and the rudder on top of the previous 2 actions (those being throttle and elevators).

B. HEADING CONTROL/NAVIGATION

Heading control in papers mostly focuses on the outer loop control of the yaw DOF angle (see Figure 9). Whereas yaw is often not directly included in attitude control tasks, it is the main focus of heading control tasks. Yaw control via the rudder is to keep the plane coordinated, while you should control heading with the ailerons. Heading control essentially means controlling where the plane goes, or where it heads. It is not overly concerned with the stability of the plane, that part is taken care of by the inner loop attitude control. When extending to full path planning navigation tasks, altitude can also be introduced into the mix (and combining attitude control and outer loop control is also possible/desirable).

1) CASE STUDY

Rennie [19], [86] describes a toolkit and methodology for heading and altitude control of fixed-wing aircraft using the PPO algorithm and JSBSim. The goal is for the agent to steadily fly along a trajectory that is set to follow a specific angle on the compass. Rennie formulated two MDPs for this task, with the second being more challenging than the first. For both cases the agent spawns at good altitude and velocity with no normal or rotational velocities and no control inputs. In the first MDP the agent is spawned with perfect heading and has to then keep the plane on it, while the second MDP spawns the agent at a random heading (all 360°) from which the agent then has to turn to achieve the target heading. The agent controls the roll, pitch and yaw of the airplane via the ailerons, rudder and elevator, with a state space of altitude, pitch, roll, yaw, linear velocities, angular velocities, the positions of the 3 control surfaces, the difference between the current and desired state (heading, altitude, roll, sideslip) and finally also with the number remaining steps in the episode. It should also be noted, while the main objective of



FIGURE 9. Image depicting an airplane top-down, showcasing how yaw affects the aircraft (“turning” left/right). Screenshot taken in GeoFS HD [100].

this work is heading control, that the agent does consider the attitude (roll) in the reward function, as the reward function uses the altitude error, heading error and also tries to keep roll and sideslip at a minimum via extra shaping. Episodes terminate in case either the number of remaining timesteps reaches 0 or when the agent falls below a minimal threshold altitude. Rennie then tested whether the same methodology could be used on 3 widely different aircraft, those being a Cessna 172P (a single engine general aviation machine) a F-15 fighter jet and an Airbus A320 (a large commercial twin-engine airliner). The experiment showed that all 3 aircraft achieved policies that are similarly competent on both MDP scenarios after training.

2) HEADING CONTROL

In heading control tasks the agent is tasked with maneuvering the plane to face a certain direction (yaw). To achieve this, the RL algorithm is given control over the aircraft’s control surfaces (ailerons, elevators, rudder) in order to utilize these to obtain the desired state. Rewards are generally given based on the deviation to the desired state and the state representation often focuses on the current attitude (plane’s rotation) and angular velocities.

Kimathi [50], [51] proposed a methodology for heading control for fixed-wing aircraft (aileron and rudder) using a SARSA algorithm approach [129]. It uses X-Plane for flight dynamics and the resulting model allowed the agent to perform at a level better than that of a PID controller, using a state representations made up of roll, yaw, heading rate, roll rate, as well as aileron and rudder stick deflections. The RL agent has control over ailerons and rudder. The reward is calculated by looking at the desired state and the current deviation from it.

Eckstein, in his thesis [83], introduces a multi-agent fixed wing controller that handles the pitch, roll and the heading to achieve gliding descent of an un-propelled fixed wing aircraft with DDPG and MADDPG [130] as well as his toolkit, that

was forked from Rennie’s repository and extended upon to support the tasks that Eckstein was aiming for.

In [131] Zhang et al. implement cruise control (keeping the aircraft at a constant altitude, velocity, heading, pitch and roll) model using DDPG. The agent was allowed to interact via the ailerons, rudder, elevators (roll, yaw, pitch) and throttle. The values considered for the state and reward were the same, those being: the altitude error, the velocity errors (x, y, z), the roll, pitch and yaw, the angular velocities, the integral of the altitude error as well as the integral of the velocity on the x and y axis.

3) PATH PLANNING

Path planning tasks via RL are often carried out in a simplified “grid” setting, where no real flight dynamics are simulated, but rather a more abstract control system of moving along a grid is implemented (but work with simulated flight dynamics can also be found). The goal for the RL agent is to move the plane across the map so that the plane will reach a set goal on it. States are often images (which is uncommon in other tasks) or the planes coordinates and the goal coordinates with other supporting data. The reward usually considers the distance of the plane at the current timestep in regards to the goal.

In [132] Yan et al. propose a navigation task for UAVs using Q-learning. This task is carried out using the STAGE scenario, meaning it does not actually consider real flight dynamics, but rather functions more like a very abstract representation of flight, in which the plane can just maneuver along a grid (within physically possible angles though). That being said, the state space is made up of a gridded version of the maneuverable map, where each bracket of the grid is a single state (in this case the grid is made up of 70×70 squares). The action space is also discrete, and includes all 8 possible squares that the plane could reach from the square it is currently in (up, down, left, right and all 4 diagonals). Lastly, the reward is calculated via the distance from the goal square with sparse rewards for maximum range away from goal, leaving the map boundaries, entering threat areas and a reward for reaching the goal square. In [133] Yan et al. expand upon this idea. Here dueling double deep Q-learning (3DQN) was used in place of simple Q-learning. STAGE is still used to emulate the plane’s flight, but instead of using discrete states via the binned grid, here the map is converted to a simplified image that is then fed into the 3DQN’s neural network. The output (actions) has remained the same (still discrete). The reward function has changed a bit, where now the distance portion of the reward has been removed and the values for the sparse rewards have been changed, however the parameters are still the same.

Zhou et al. describe a 3D path finding agent in [134]. The agent is presented with a target location in 3D space that it has to reach, with the 3D space also having mountain like elevations and obstacles. For that, the SAC algorithm was used with an input state of size 35 (relative position of the agent (to elevation), relative position of the target (in relation to the agent), relative position of the nearest obstacle, range

ratio (how long plane can keep flying), horizontal heading angle, and the local elevation data (26 data points)). The actions that the agent can control are the turning angle and the pitch angle. The reward was shaped via the change in distance to the goal, change in distance to obstacles, and the altitude.



FIGURE 10. Image depicting flocking with a leader (yellow) and 2 following planes (red). The circle depicts the desirable area around the leader for the following planes to be in. Screenshot taken in GeoFS [100].

C. FLOCKING

Flocking, in the context of aircraft, is referring to formation flying. It can somewhat be thought of similar to how swarms of birds fly in close proximity. Just like bird swarms, when flocking, aircraft fly in close proximity to one another (see Figure 10). This can have advantages such as better fuel consumption or aerodynamics as well as increase the mission capabilities of the group of aircraft. When talking particularly about UAV, then flocking also means that these multiple UAVs work together in a coordinated fashion based on the other planes to achieve a goal in the group, often in a leader-follower hierarchy.

1) CASE STUDY

Hung et al. [135], [136] employ variable learning rate reinforcement learning (Pongs $Q(\lambda)$ ($PQ(\lambda)$) [137] with Dyna architecture) to control multiple small fixed-wing aircraft. In the experiment setup a plane was classified as the leader with other planes tasked with following that leader, who is following a separate set of instructions and is not being trained. In training, each agent was following a single leader. Agents are also not object to collision, as they are operating on different altitudes (altitudes as well as speed are fixed). The state space used here is only focused on the leader follower relation and is therefore defined by the differences between the x and y (which are the planar positions of the plane) as well as the heading and roll of leader and follower, and lastly the leaders roll command. For the action space the roll angle setpoint will be chosen by the agent and then given to and regulated by the auto-pilot. Actions are also set up so that any action can only deviate to a certain extent from the previous action. The reward is shaped in a way where

agents are incentivised to be near but not too near to the leader (being to close and too far away is discouraged by the reward function). As stated before, followers do not interfere with each other as they are considered to be on separate altitude levels and therefore agents also do not consider each other, but rather only focus on themselves and the leader. Due to the nature of this setting experiments can be run with very large numbers of followers at once and since all followers share a single Q-table, the overhead is also limited. Each agent would spawn at a random position. The shared Q-table also speeds up the learning process, as one timestep now updates the Q-table once for each agent. An episode was capped to 30 timesteps with each timestep equalling a second of flight time. Episodes would also terminate when an agent would leave the maximum acceptable range of the leader. Results have proven to be near optimal quintero's dynamic programming (QDP) [138].

2) FLOCKING

RL agents need to primarily concern themselves with themselves, the leader and in some works also the other followers. Often altitude is not part of the RL agents duties so it most often is responsible of controlling the aircraft's roll (and often also velocity) so that it remains within distance of the leader. The reward is usually shaped in a way where agents receive more reward the closer they are to the leader and inputs are often concerned with the agents current state and the state of the leader (or the difference between the two).

In [139] Wang et al. propose and use the Continuous actor-critic with experience replay (CACER) reinforcement learning algorithm with double prioritized experience replay (DPER) to train their agents. The problem statement is kept in accordance with Hung et al. and therefore also deals with a single leader and followers at different attitude (here too the altitude and speed it constant). The leader also follows another goal and does not learn via RL and also shares his information with the other planes so they can compute their states and rewards. Agents are also only in control of the roll, through the autopilot. State, reward and actions are also kept in accordance with the previous work, but without having to discretize them, due to CACER being a deep RL algorithm. One more addition that this paper added was a semi-physical test integration. In later work Wang and Yan et al. improved upon this base and removed the limitation that all planes are flying at a constant speed in [140]. The RL algorithm used was a version of continuous actor-critic learning automation (CACL) [141], [142] that was specifically modified for this flocking problem. The state space includes the planar position difference between the leader and follower, the difference in the two's heading, the leaders roll and velocity setpoint, the leader's actual roll and velocity and the followers roll and velocity. The controllable values are velocity and roll and the reward is given based on the distance to the leader. Next, in [143] the limitation of having all planes operate on fixed different altitudes was removed and planes now also had to consider collision avoidance during flocking as all planes are

operating on the same altitude in this work. In this paper the authors introduce the PS-CACER (no name was given, but we believe it stands for policy sharing CACER), an extension to the CACER algorithm that allows multi-agent scenarios. The paper also introduces SEMP, an embedding method that allows for inputs of variable length. The state space is of variable length, as it now also includes the other follower's information (in addition to one's own and the leader's), as not to collide with them (whereas before a follower only had to consider itself and the leader). With that the state is the same way as it was in the previous paper (for the follower-leader related part nothing changed in the state space), but on top of all the values from the previous paper the state now includes the relative location between the "ego"-follower and another follower i , as well as follower i 's and the "ego"-follower's difference in heading and follower i 's roll and velocity. This secondary vector will be added n times, for n other followers, hence the variable length. The same is also true for the reward, the first half of the reward function is identical ("ego"-follower and leader), but a second portion has been added on top, the one concerning followers i . The second half gives rewards based on the distance between the "ego"-follower and follower i , making sure they don't get too close or even collide. The action space however has remained the same. In [144] Yan et al. propose multi-agent adaptive D3QN (MA2D3QN), a new deep RL algorithm to flocking. This paper is also concerned with collision avoidance in flocking. In this paper 3 modules are working together to achieve the tasks, those being a situation assessment module, an action selection module and the D3QN. The situation assessment module calculates the risk of collision for each follower based on how close they are to other planes. For this module the state space for "ego"-follower-leader has remained the same. This model will forward the state (to the action selector and the D3QN) and a map that depicts the crash risk per plane for all planes (only to the D3QN). The D3QN will take the collision probability map input and generate Q-values for it (for roll and velocity), so it will function as the Q-value function approximator here. It will take the state as described above and the map as input and generate Q-values accordingly. These Q-values are then forwarded to the action selection module, which also receives the state from the situation assessment module. Using these two value pairs as inputs it will return the final action values. Through MA2D3QN this can be done in a multi-agent setup, where all followers can learn at the same time on the same model. Lastly, Yan et al. added obstacles to the environment in [145].

A leader-follower flocking approach with one leader and five followers is proposed in [146] by Xu et al. Here the authors use PPO, or PPO-Exp to be more exact (a modification they made to improve exploration performance). The model is trained in a way where the "brain" (the model) is located in the leader alone (centralized approach). This means that the followers are not capable of action alone, as they receive their instructions from the leader and send their state information back to the leader so that the central model can

use it to predict actions for all planes. The state-space is made up of the x and y coordinates, the heading, roll, velocity and angular roll velocity (all values six times, one for each plane). The action space allows each plane to control the linear acceleration (thrust) and angular acceleration (roll). For rewards, the model uses the distance between the followers current position and its target position within the group, the distance between the follower and the leader for all followers, the distance of the leader to the goal location as well as the distance in between the followers to avoid collisions. The authors also conducted tests of flocked obstacle avoidance.

D. LANDING

Landing might be the most self explanatory of the tasks listed in this paper and also one of the most important ones. While heading control is important to make sure that the plane actually ends up in the right place, landing makes sure that it does so in one piece. RL research has been conducted in this field and it should also be noted that this is one area where the fixed-wing problem of landing and the quadcopter problem of landing differ very immensely (maybe the most), with the fixed-wing landing procedure being arguably more difficult and dangerous.

1) CASE STUDY

In [18] Waldock et al. introduce an approach to perched landing using DQN. Perched landing is a special sub-case within landing, where unlike commercial planes landing by approaching a runway and coming to a stop on the gears after touchdown, in perched landing, aircraft go into stall on purpose to essentially stand still midair, to then drop straight down to the point where they plan to land. For this a variable sweep wing aircraft was utilized, an aircraft that can sweep its wings forwards and backwards. This helped in achieving a stalling state. In this work the authors built their own numerical model that was based on an off-the-shelf remote controlled airplane frame that was modified in a way that allows it to sweep its wings. The plane was then tested in a wind tunnel test setting to obtain accurate data. The state space transmitted to the DQN contains 14 values, those being: the aircraft's position (x, y, z), the attitude, aircraft's speed on all 3 axis, the 3 angular velocities, the delta sweep and the delta elevator angle. The actions (49) are combinations of the rate at which the variable sweep wings and elevators change. This paper compared multiple different reward functions. The first one only considered speed and position, the second one added control inputs, the third added the pitch as well (which did the best). Experiments were carried out over a total of 5 million steps, with each timestep being 0.1 seconds of flight time. Noteworthy is also the fact that the modified model airplane was later used to conduct real world testing.

2) PITCH ONLY

Pitch only landing requires the RL agent to achieve appropriate pitch that would allow the plane to perform a landing maneuver. To achieve this the agent is given control over the aircraft's pitch.

Yuan et al. use deterministic policy gradient (DPG) to explore fixed-wing landing in [147].

[148] Tang et al. use DDPG to train a landing agent. The agent controls the pitch angle of the airplane via the state of altitude, altitude error, altitude rate, altitude rate error, and the aircraft's position. The reward is calculated through the altitude deviation, the altitude rate deviation and a touchdown reward value.

In [149] Zahmatkesh continued their research using fuzzy Q-learning (FQL) [150]. The state space is the same as it was in [118], as is the action space. The state space is made up of the pitch angle and the pitch angular velocity, and the action state is the control of the planes pitch axis. The reward (also identical) calculates the amount at which the elevator is actuated, as well as the difference between current and desired state of the plane. A path planning module will send the RL module the desired pitch at time-step t , which the module will try to achieve (similar to pitch only attitude control, but with a desired state that keeps adjusting).

3) PERCHED LANDING

In perched landing the agent is also tasked with controlling the pitch, but also needs to control the sweep angle of the sweep wings in order to get the agent so stall at the right time to reduce velocity and to achieve the conditions needed to land.

Fletcher et al. utilize PPO for perched landing in [151], following up on Waldock et al.'s work. Experiments were carried out over 3 different test cases, with different levels of difficulty through wind modelling. The action space allowed the agent to control the elevator and sweep angles (7 discrete values for each, resulting in 49 action combinations). The state space includes the planes position (x,y,z), the pitch angle, the planes longitudinal velocities, pitch rate, and the elevator and sweep angles. The reward function used here compares the actual final state to the desired final state (x, z, pitch angle, longitudinal body velocities). In a follow up paper, [152] Fletcher et al. extend the agent by allowing it to also control the throttle of the airplane, to make up for limitations in the previous paper where the plane had difficulties in heavy winds. The way the throttle was added to the agent's action space was binary, so the agent can either engage or disengage the throttle, but not control how much so. The state space remains the same, but with the addition of values that show whether the throttle is on or not. The reward function remains the same.

E. TAXIING

Taxiing is what on the ground maneuvering of aircraft is called. This for example includes the aircraft making its way from the gate to the runway to take off. Such tasks have not seen too much work yet, but they have seen work nonetheless.

1) CASE STUDY

In [153] Zhang et al. present a taxiing agent (front wheel driven aircraft) that was trained via PPO. The agent is in

control of a F-16 "Falcon" jet that the author modelled for this work. To accomplish the task of taxiing the agent was presented a state of linear velocities (x,y,z), angular velocities (x,y,z), the x and y coordinates of the upcoming 5 points on the taxiing trajectory, the steering angle of the wheel, as well as the rotary velocity of all 3 wheels, resulting in a state space size of 20. The action space contains the steering angle and the towing torque of the wheel's motor. The reward considers wheel slip, velocity, distance to checkpoints and closeness to the trajectory. The experiment is set up in a way in which both the target trajectories as well as the spawning conditions of the plane are random, so that the agent will get exposed to a variety of different states during training, in order to give it higher robustness. An episode lasts for 1000 timesteps, each of which lasts for 0.05 seconds and the agent was trained for over 600,000 such episodes. The final agent can follow the trajectory well and does not deviate from it until it reaches the goal.

2) TAXIING

In taxiing the RL agent is tasked with maneuvering the plane in a taxi maneuver from its current location to a designated goal.

Xiang et al. use tabular Q-learning to explore taxiing in [154]. Here the problem is boiled down to a grid, similar to the well known frozen lake problem. Agents need to traverse this gridded (10 × 10) airport to get from point A to point B, where the ratio of passable and impassable cells varied. Moving obstacles were also considered during testing. By nature of this being a gridded representation, the state is represented by said grid and the actions are up, down, left and right within the grid environment. As reward, the agent gets penalized for every timestep as well as colliding with an obstacle and gets positive reinforcement for reaching the goal.

F. OTHERS

In this section we will look at papers that do not necessarily fit into one of the above sections, either by incorporating multiple of them into one, or by doing something totally different altogether.

[155] Chang et al. combine RL controlled UAVs and a human controlled manned aerial vehicle (MAV) to collaborate on a flocking + path planning task.

In [156] a training environment is described that trains agents to control pitch, roll, heading and the level in order to avoid collisions between the trained plane and other planes.

In [157] the authors formulate a methodology to allow their PPO agent to perform a stall maneuver, meaning their agent aims to forcefully enter a stalled state mid-flight, to then escape that state again to continue on flying. The agent has control over the throttle, elevator, aileron and rudder to achieve that task and receives a state that includes the difference in altitude with the desired altitude, the error in velocity with the desired one, the roll angle, pitch angle, yaw angle, angle of attack, angle of sideslip and the angular velocities.

TABLE 3. Legend of Tables 5 and 6.

—	undisclosed
Δ	distance between current state and desired state
α	Angle of Attack
β	Angle of Sideslip
a	acceleration
aa	aspect angle
ac	Attitude Control
af	approach & follow
alr	altitude rate
alt	altitude
ap	all planes
as	airspeed
av	angular velocities
ba	bank angle
c	continuous
ca	climb angle
cc	cruise control
cs	control surface
cp	checkpoint
d	discrete
dist	planar position difference
dem	terrain elevation data
e	errors
f	follower
fc	fluid coefficients
f_i	Follower i
fl	flocking
flo	follower, leader, obstacles states
g	grid
gi	grid as image
gl	goal
gm	grid movement
h	heading difference
hc	Heading Control
hca	heading cross angle
l	leader
lc	longitudinal control
lan	landing

The agent receives rewards that are calculated with the roll, pitch and yaw angles as well as the altitude and the velocity.

Ou et al. train an obstacle avoidance agent in [158] using SAC. To ensure stable flight, the agent relies on a inner loop attitude control module, which is however not trained via RL, but rather uses a low level attitude controller. The agent itself can therefore worry about the obstacle avoidance task solely. The agent is therefore mostly concerned with line of sight and loss of altitude. The input state vector is the obstacles relative x and y to the plane, the altitude, roll, pitch, yaw, the velocity on the x and y axis, the line of sight (which is the angle between the agents heading and the obstacle) to the obstacle and the minimum line of sight angle needed to avoid it. The actions are the control of the roll, pitch, yaw and throttle and the reward considers the line of sight, the minimum line of sight, the yaw as well as the loss of altitude.

In [159], Wu et al. use the smoothness of fluid dynamics to their advantage to create smooth obstacle avoidance paths for their aircraft. The agent was trained using DDPG and

TABLE 4. Continuation of Table 3.

lv	linear velocities
loa	loss of altitude
los	line of sight
lov _{u,v,w}	longitudinal velocities
mt	multi-task
nz	non-zero angles
o	overshoot
oa	obstacle avoidance
obs	obstacle
ot	overtake
p	pitch
pa	previous action
pc	pitch control
pl	perched landing
pp	path planning
r	roll
rv	rotary velocities of wheels
s	sparse
sf	success flag
set	setpoint
sl	slider
sm	stalled maneuver
sr	additional sparse rewards
sw	sweep
t	throttle
ta	target
td	touchdown
tj	trajectory
tq	torque
ts	per timestep
tr	timesteps remaining
tx	taxiing
ut	upcoming targets (x, y)
v	velocity
ws	wind speed
wsa	wheel steering angle
wsl	wheel slip
(x, y, z)	aircraft position
y	yaw

did not control the aircraft's control surfaces directly, but rather outputs 3 coefficients that influence the fluid dynamics which then based on these coefficients predict a flight path that in turn is then executed by the plane. The state space that the agent receives is the delta between the obstacle and the aircraft's position (x,y,z) as well as the delta between the two's velocities on the x , y and z axis. It includes the distance between the obstacle and the plane, the climbing angle, angle of attack (AoA), banking angle and lastly also the airspeed. The reward takes into consideration the distance to the obstacle, the distance to the goal position and the bank angle, climbing angle and heading angle.

[160] Choi et al. introduce a multi task agent that first approaches another plane and then tracks and follows said plane's maneuvers from there on out using SAC and JSBSim. The state-space contains: (x,y,z) , the velocity on the x , y and z axis, the roll, pitch and yaw angle, the 3 angular velocities, distance between UAV and target on x , y and z , the pitch

TABLE 5. Table summarizing the papers in the fields as discussed in Section IV. This table aims to give a quick overview of the work presented in this paper, but has to simplify the contents by way of limited space. For more detailed information look at Section IV or the papers themselves. Papers where too much of the information that is being shown in this table is not mentioned have been omitted. If multiple papers use the same parameters, then only one will be listed here (but all will be listen in the full-text, in Section IV). The legend explaining all abbreviations can be found in Tables 3 and 4.

Ref	Task	Authors	Year	RL	State	Action	Reward
[106]	pc	Monaco et al.	1997	RAL	—	d: p	—
[115]	pc	Huang et al.	2019	DDQN	c: p , av	d: p	Δ , pa
[116]	pc	Wada et al.	2021	A3C	c: Δ , α , av, cs, ws	d: p	α
[118]	pc	Zahmatkesh et al.	2022	QL	d: p , av	c: p	p , pa
[119]	lc	Chowdhury et al.	2022	PPO	c: as, r , p , y , av, lov, $a_{x,y,z}$, $t_{as,r,p}$	c: t , p	e_{as} , e_p , nz_p , p , cs
[13]	ac	Bøhn et al.	2019	PPO	c: p , r , av, as	c: r , p , y , t	Δ
[14]	ac	Zhen et al.	2020	PPO	c: p , r , y , av, as	c: p , r , t	Δ , as
[122]	ac	Richter et al.	2021	QL	d: p , r	d: p , r	Δ
[125]	ac	Li et al.	2022	DDPG	c: p , r , y , av, as	c: p , r	Δ , o
[127]	ac	Qui et al.	2022	DDPG	c: Δ	c: p , r , sl	Δ , pa
[124]	ac	Richter et al.	2022	DDQN	c: Δ , av, α , β	d: p , r	Δ
[102]	ac	Bøhn et al.	2023	SAC	c: p , r , α , β , av, as, pa, e	c: r , p	s
[128]	ac	Chowdhury et al.	2024	PPO	c: as, r , p , y , av, lov, $a_{x,y,z}$, $t_{as,r,p}$	c: t , p , r , y	e_{as} , p , r , nz_r , p , y , cs, av
[50]	hc	Kimathi et al.	2017	SARSA	c: r , y , av_r , av_y , cs_r , cs_y	d: r , y	Δ
[19]	hc	Rennie et al.	2018	PPO	c: alt, r , p , y , lv, av, cs, Δ , tr	c: r , p , y	Δ_r , Δ_h , r , β
[131]	cc	Zhang et al.	2021	DDPG	c: e_{alt} , $v_{x,y,z}$, r , p , y , av, $\int e_{alt}$, $\int e_{v_x,v_y}$	c: t , r , p , y	e_{alt} , $v_{x,y,z}$, r , p , y , av, $\int e_{alt}$, $\int e_{v_x,v_y}$
[132]	pp	Yan et al.	2018	QL	d: g	d: gm	dist, sr
[133]	pp	Yan et al.	2020	3DQN	c: gi	d: gm	sr
[134]	pp	Zhou et al.	2024	SAC	c: (x, y, z) , (x_{gl}, y_{gl}, z_{gl}) , y , tr, dem, (x_{obs}, y_{obs})	c: p , y	dist $_g$, dist $_{obs}$, alt
[135]	fl	Hung et al.	2015	PQ(λ)	d: dist $_l$, h , r_f , r_l , r_{set}	d: r	dist $_l$
[139]	fl	Wang et al.	2019	CACER	c: dist $_l$, h , r_f , r_l , r_{set}	c: r	dist $_l$
[140]	fl	Yan et al.	2020	CACLA	c: v_l , r_l , v_f , r_f , v_{set} , r_{set} , h , dist $_l$	c: r , v	dist $_l$
[143]	fl	Yan et al.	2021	PS-CACER	c: v_l , r_l , v_f , r_f , v_{set} , r_{set} , h_l , dist $_l$, c_{f_i} , v_{f_i} , dist $_{f_i}$, h_{f_i}	c: r , v	dist $_l$, dist $_{f_i}$
[144]	fl	Yan et al.	2021	MA2D3QN	c: v_l , r_l , v_f , r_f , v_{set} , r_{set} , h , dist $_l$	c: r , v	dist $_l$, dist $_{f_i}$
[146]	fl	Xu et al.	2022	PPO	c: ap(x, y, h, r, v, av_r)	c: ap(t, r), r	flo, dist $_{f_i}$, gl, ta
[148]	lan $_p$	Tang et al.	2020	DDPG	c: alt, e_{alt} , alr, e_{alr} , (x, y, z)	c: p	Δ_{alt} , Δ_{alr} , td
[149]	lan $_p$	Zahmatkesh et al.	2023	FQL	d: p , av	c: p	p , pa
[18]	pl	Waldock et al.	2018	DQN	c: (x, y, z) , r , p , y , lv, av, sw, cs_y	d: sw, y	as, (x, y, z) , $(cs, (p))$
[151]	pl	Fletcher et al.	2021	PPO	c: (x, y, z) , r , p , y , lv, av, sw, cs_y	d: sw, y	(x, y) , p , lov $_{u,w}$
[152]	pl	Fletcher et al.	2022	PPO	c: (x, y, z) , r , p , y , lv, av, sw, cs_y , t	d: sw, y , t	(x, y) , p , lov $_{u,w}$
[153]	tx	Zhang et al.	2021	PPO	c: lv, av, ut $_5$, wsa, rv $_3$	d: wsa, tq	dist $_{cp}$, wsl, lv, tj
[154]	tx	Xiang et al.	2023	QL	d: gm	d: g	g , obs, ts
[158]	oa	Ou et al.	2022	SAC	c: obs $_x$, obs $_y$, alt, r , p , y , v_x , v_y , los, los $_{min}$	c: r , p , y , t	los, los $_{min}$, loa
[159]	oa	Wu et al.	2022	DDPG	c: $\Delta_{obs(x,y,z)}$, $\Delta_{obs(v_x,v_y,v_z)}$, dist $_{obs}$, ca, h , α , ba, as	c: fc	$\Delta_{obs(x,y,z)}$, $\Delta_{obs(v_x,v_y,v_z)}$, dist $_{obs}$, ca, h , α , ba, as
[160]	af	Choi et al.	2023	SAC	c: (x, y, z) , v_x , v_y , v_z , r , p , y , av, dist $_l$, x , dist $_l$, y , dist $_l$, z , p_l , h_l , dist $_l$, aa, hca	c: s_x , s_y , s_z , r , t	dist $_{gl}$, alt, h , gl, los, tr, ot

angle to the target, heading angle to the target, absolute distance to the target, aspect angle and heading cross-angle. The actions are the x and y axis movement of the stick, the throttle and the rudder controls. In the approach task the

agent receives reward when getting closer, having similar attitude and heading and when reaching the other plane and receives penalties when crashing, overtaking the other plane and when out of time steps. The second task also gives reward

TABLE 6. Continuation of Table 5.

Ref	Task	Authors	Year	RL	State	Action	Reward
[157]	sm	Hu et al.	2022	PPO	$c: \Delta_{alt}, \Delta_v, r, p, y, \alpha, \beta, av$	$c: t, r, p, y$	r, p, y, alt, v
[161]	mt	Li et al.	2023	PPO	$c: av, lv, r, p, tr, cs, t, e_h, e_{alt}, sf$	$c: \int r, \int p, \int y, \int t$	$\Delta_{alt}, \Delta_h, \Delta_r$

for successfully reaching the goal, in addition to line of sight, closeness, as well as an additional score for reaching a desirable location close to the goal. Punishment is again given for overtake, crashing and running out of time.

Li et al. [161] propose a methodology that is capable of executing five different flight maneuvers (level flight, coordinated turn, climb, descent, horizontal roll) in their paper, the first agent to be able to do that according to the authors. Each maneuver is however trained individually. The agent uses PPO to learn. The action space gives the agent control over the roll, pitch, yaw, throttle, but uses the integral of the models output, as opposed to using the values as they are, offering smoother controls. The state space includes the angular velocities, linear velocities, roll, pitch, the remaining time steps, the current control surface positions (aileron, elevator, rudder), current position of the throttle, the heading error and the error in altitude. However, when learning the task of horizontal roll, the agent no longer receives the heading error, but instead now receives a flag of whether or not it has completed a full turn (360 degrees) and the roll error as additional states. The reward function is also considerate of which of the 5 tasks the agent is currently attempting. For level flight, climb, descent and coordinated turn, altitude delta and heading delta are used, but for the rolling task the agent also considers the roll data to create a smoother maneuver. Additionally the values are weighted differently based on which task is being performed as well.

V. DISCUSSION & POSSIBLE DIRECTIONS

One contribution that could greatly help this area of research would be the development of a universally applicable metric for some of the more common tasks, such as attitude control for example. Most papers, even though they attempt very similar tasks through different methodologies, have no directly comparable metric that would allow for direct comparison and while that is not necessarily a huge problem, especially in these earlier stages and in the “feasibility study” stage that we still seem to be in, it would be very welcome if later on a metric would be constructed that would measure the improvement one made when compared to older research. Something else that is rather common nowadays is the use of various different flight simulators and toolkits, which has both pros and cons. If ever a toolkit would establish itself as the go-to for this field, then that could also enable simpler benchmarking, even though many toolkits are already implemented in a way where they support OpenAI Gym and with that also the newer, and still maintained, fork “Gymnasium” [162], which already simplifies things a lot.

Another consideration would be to allow AI to take more control of the learning process, where humans would take

over more of a feedback role, as opposed to crafting all elements of the RL pipeline themselves as in [74], [75], [76], and [77], in line with the trend of shifting more and more to data-driven ML and DL [78].

One other possibility would be the development of an all encompassing model that can not just complete one, but rather the full range of tasks necessary to complete a flight (or a handful of tasks) without human intervention or assistance, a fully-automated system so to speak. One that can handle taxiing, air traffic control communications, take-off, heading control/navigation, attitude control, landing and everything else in between.

Lastly, another area that, understandably so, has yet to see too much work is the area of working with real physical planes, be it models or full-size ones, instead of working with simulators, both for training or for testing. Obviously, this comes with a much higher risk, as unsuccessful runs would be very costly and potentially also very dangerous, but of course eventually this has to become the goal towards which researchers want to strive.

VI. CONCLUSION

While reinforcement learning for fixed-wing aircraft is still a niche research interest, it has picked up steam in recent years and made good progress along the way. Papers ranging from single DOF control using a real model to papers that control all 6 DOF in simulation. From papers that cover the taxiing task to papers that handle outer loop control. Papers that use traditional RL methodology to papers that use state of the art, deep learning enhanced, deep reinforcement learning algorithms. RL for fixed-wing aircraft has become a research topic with a wide range of different subsections, all of which have their own discoveries to be made. This paper attempts to give an up-to-date overview of the current state of the fixed-wing RL aircraft control landscape (focusing on attitude control, heading control, flocking, landing taxiing, etc.), giving a brief introduction to each of the fields and papers, including the relevant information for researchers to decide whether the paper would be relevant to them. Reading this paper will not replace reading the papers of interest in full, but we attempt to give people interested in entering the fixed-wing RL landscape a simple point of entry to see where the field currently stands and to offer an overview hub for finding relevant work in the field one might be interested in. While many works that focus on the same task do follow similar approaches, currently fixed-wing RL has no standard frameworks, toolkits, methods or metrics, which makes it difficult to compare the performance of different papers. This is something that could be explored in future works. Human feedback based RL might be another direction

that researchers could look towards, as it would remove the challenge of formulating hand crafted reward functions and fine tuning them. In its current state, fixed-wing RL already produces strong policies and shows its potential and capabilities to be applied in these areas, laying the ground work for even stronger, more robust, better generalizing and more encompassing works in the future.

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [2] V. Mnih, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [3] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, May 1992.
- [4] G. Tesauro, "Temporal difference learning and TD-Gammon," *Commun. ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [5] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
- [6] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, Oct. 2017.
- [7] A. E. Sallab, M. Abdou, E. Perot, and S. Yogamani, "Deep reinforcement learning framework for autonomous driving," *Electron. Imag.*, vol. 2017, no. 19, pp. 70–76, 2017.
- [8] G. Lample and D. S. Chaplot, "Playing FPS games with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, no. 1, pp. 1–7.
- [9] H.-N. Quach, S. Yeom, and K. Kim, "Survey on reinforcement learning based efficient routing in SDN," in *Proc. 9th Int. Conf. Smart Media Appl.*, 2020, pp. 196–200.
- [10] A. P. Patil, S. Raghavendra, S. Srinarasi, and R. Ram, "A deep reinforcement learning approach to the ancient Indian game—Chowka Bhara," *TechRxiv*, Oct. 2021, doi: [10.36227/techrxiv.16780414.v1](https://doi.org/10.36227/techrxiv.16780414.v1). [Online]. Available: <https://www.techrxiv.org/doi/full/10.36227/techrxiv.16780414.v1>
- [11] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conf. Robot Learn.*, 2022, pp. 91–100.
- [12] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement learning for UAV attitude control," *ACM Trans. Cyber-Phys. Syst.*, vol. 3, no. 2, pp. 1–21, Apr. 2019.
- [13] E. Böhn, E. M. Coates, S. Moe, and T. A. Johansen, "Deep reinforcement learning attitude control of fixed-wing UAVs using proximal policy optimization," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2019, pp. 523–533.
- [14] Y. Zhen, M. Hao, and W. Sun, "Deep reinforcement learning attitude control of fixed-wing UAVs," in *Proc. 3rd Int. Conf. Unmanned Syst. (ICUS)*, Nov. 2020, pp. 239–244.
- [15] N. Parvez Farazi, B. Zou, T. Ahamed, and L. Barua, "Deep reinforcement learning in transportation research: A review," *Transp. Res. Interdiscipl. Perspect.*, vol. 11, Sep. 2021, Art. no. 100425.
- [16] H. Kim, M. Jordan, S. Sastry, and A. Ng, "Autonomous helicopter flight via reinforcement learning," in *Proc. NIPS*, vol. 16, 2003, pp. 1–8.
- [17] Y.-J. Chen, D.-K. Chang, and C. Zhang, "Autonomous tracking using a swarm of UAVs: A constrained multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13702–13717, Nov. 2020.
- [18] A. Waldock, C. Greatwood, F. Salama, and T. Richardson, "Learning to perform a perched landing on the ground using deep reinforcement learning," *J. Intell. Robot. Syst.*, vol. 92, no. 3, pp. 685–704, 2018.
- [19] G. Rennie, "Autonomous control of simulated fixed wing aircraft using deep reinforcement learning," M.S. thesis, Dept. Comput. Sci., Univ. Bath, Bath, U.K., 2018. [Online]. Available: https://purehost.bath.ac.uk/ws/portalfiles/portal/216919613/Rennie_Gordon.pdf
- [20] C. Wang, J. Wang, Y. Shen, and X. Zhang, "Autonomous navigation of UAVs in large-scale complex environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2124–2136, Mar. 2019.
- [21] Airbus-AI-Research. (2020). *Scikit-Decide—AI Framework for Reinforcement Learning, Automated Planning and Scheduling*. [Online]. Available: <https://github.com/airbus/scikit-decide>
- [22] (2020). *Scikit-Decide—AI Framework for Reinforcement Learning, Automated Planning and Scheduling*. [Online]. Available: <https://airbus.github.io/scikit-decide/>
- [23] P. Abichandani, C. Speck, D. Bucci, W. McIntyre, and D. Lobo, "Implementation of decentralized reinforcement learning-based multi-quadrotor flocking," *IEEE Access*, vol. 9, pp. 132491–132507, 2021.
- [24] K. Pattanayak, V. Krishnamurthy, and C. Berry, "Inverse-inverse reinforcement learning. how to hide strategy from an adversarial inverse reinforcement learner," in *Proc. IEEE 61st Conf. Decis. Control (CDC)*, Dec. 2022, pp. 3631–3636.
- [25] T. Singliar and D. D. Margineantu, "Scalable inverse reinforcement learning via instructed feature construction," in *Proc. Workshops 26th AAAI Conf. Artif. Intell.*, 2012, pp. 1–3.
- [26] T. C. Staudinger, Z. D. Jorgensen, and D. D. Margineantu, "XPlane-ML—an environment for learning and decision systems for airplane operations," 2018. [Online]. Available: <https://openreview.net/revisions?id=S1IR6YT4nQ>
- [27] V. Agarwal and R. R. Tewari, "Attitude control in unmanned aerial vehicles using reinforcement learning—A survey," in *Congress on Intelligent Systems (Advances in Intelligent Systems and Computing)*, vol. 1335, H. Sharma, M. Saraswat, A. Yadav, J. H. Kim, and J. C. Bansal, Eds., Singapore: Springer, 2021, pp. 495–510, doi: [10.1007/978-981-33-6984-9_40](https://doi.org/10.1007/978-981-33-6984-9_40). [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-33-6984-9_40#citeas
- [28] A. Carrio, C. Sampedro, A. Rodriguez-Ramos, and P. Campoy, "A review of deep learning methods and applications for unmanned aerial vehicles," *J. Sensors*, vol. 2017, no. 1, 2017, Art. no. 3296874.
- [29] A. T. Azar, A. Koubaa, N. Ali Mohamed, H. A. Ibrahim, Z. F. Ibrahim, M. Kazim, A. Ammar, B. Benjdira, A. M. Khamis, I. A. Hameed, and G. Casalino, "Drone deep reinforcement learning: A review," *Electronics*, vol. 10, no. 9, p. 999, 2021.
- [30] P. J. D. de Oliveira Evald, V. M. Aoki, C. B. da Silva, D. S. Cardoso, P. M. Pinheiro, S. S. da Costa Botelho, and P. L. J. D. Junior, "A review on quadrotor attitude control strategies," *Int. J. Intell. Robot. Appl.*, vol. 8, pp. 230–250, Jan. 2024.
- [31] J. Jagannath, A. Jagannath, S. Furman, and T. Gwin, "Deep learning and reinforcement learning for autonomous unmanned aerial systems: Roadmap for theory to deployment," in *Deep Learning for Unmanned Systems (Studies in Computational Intelligence)*, vol. 984, A. Koubaa and A. T. Azar, Eds., Cham, Switzerland: Springer, 2021, doi: [10.1007/978-3-030-77939-9_2](https://doi.org/10.1007/978-3-030-77939-9_2). [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-77939-9_2#citeas
- [32] A. Koubaa and A. T. Azar, *Deep Learning for Unmanned Systems*. Cham, Switzerland: Springer, 2021. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-030-77939-9>
- [33] F. AlMahamid and K. Grolinger, "Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review," *Eng. Appl. Artif. Intell.*, vol. 115, Oct. 2022, Art. no. 105321.
- [34] L. Mao, X. Wu, Z. Huang, and A. J. Tatem, "Modeling monthly flows of global air travel passengers: An open-access data resource," *J. Transp. Geography*, vol. 48, pp. 52–60, Oct. 2015.
- [35] K. Dismukes, *Human Error in Aviation*. Farnham, U.K.: Ashgate, 2009.
- [36] E. Salas and D. Maurino, *Human Factors in Aviation*. New York, NY, USA: Academic, 2010.
- [37] L. Li, X. Zhang, Y. Wang, C. Qian, and M. Zhao, "Reinforcement learning methods for fixed-wing aircraft control," in *Proc. 9th Int. Conf. Syst. Inform. (ICSAI)*, Dec. 2023, pp. 1–8.
- [38] L. P. Osco, J. M. Junior, A. P. M. Ramos, L. A. de Castro Jorge, S. N. Fathollahi, J. de Andrade Silva, E. T. Matsubara, H. Pistori, W. N. Gonçalves, and J. Li, "A review on deep learning in UAV remote sensing," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 102, Oct. 2021, Art. no. 102456.
- [39] S. Samaras, E. Diamantidou, D. Ataloglou, N. Sakellariou, A. Vafeiadis, V. Magoulinitis, A. Lalas, A. Dimou, D. Zarpalas, K. Votis, P. Daras, and D. Tzovaras, "Deep learning on multi sensor data for counter UAV applications—A systematic review," *Sensors*, vol. 19, no. 22, p. 4837, Nov. 2019.

- [40] H. Kurunathan, H. Huang, K. Li, W. Ni, and E. Hossain, "Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 496–533, 1st Quart., 2024.
- [41] O. Katsuhiko, *Modern Control Engineering*. Miami, FL, USA: Félix Varela, 2009.
- [42] G. F. Franklin, J. D. Powell, A. Emami-Naeini, and J. D. Powell, *Feedback Control of Dynamic Systems*, vol. 4. Upper Saddle River, NJ, USA: Prentice-Hall, 2002.
- [43] J. Macki and A. Strauss, *Introduction to Optimal Control Theory*. New York, NY, USA: Springer-Verlag, 1982. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1356114>
- [44] R. C. Dorf and R. H. Bishop, *Modern Control Systems*, 12th ed., Englewood Cliffs, NJ, USA: Prentice-Hall, 2011. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7265050>
- [45] B. C. Kuo and M. F. Golnaraghi, *Automatic Control Systems*, vol. 8. Englewood Cliffs, NJ, USA: Prentice-Hall, 1995.
- [46] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*. Research Triangle Park, NC, USA: Instrument Society of America, 1995. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/1453566>
- [47] G. Sudha and S. Deepa, "Optimization for PID control parameters on pitch control of aircraft dynamics based on tuning methods," *Appl. Math. Inf. Sci.*, vol. 10, no. 1, p. 343, 2016.
- [48] S. Deepa and G. Sudha, "Longitudinal control of aircraft dynamics based on optimization of PID parameters," *Thermophys. Aeromech.*, vol. 23, no. 2, pp. 185–194, 2016.
- [49] T. Susanto, M. Bayu Setiawan, A. Jayadi, F. Rossi, A. Hamdhi, and J. Persada Sembiring, "Application of unmanned aircraft PID control system for roll, pitch and yaw stability on fixed wings," in *Proc. Int. Conf. Comput. Sci., Inf. Technol., Electr. Eng. (ICOMITEE)*, Oct. 2021, pp. 186–190.
- [50] S. Kimathi, "Application of reinforcement learning in heading control of a fixed wing UAV using X-plane platform," *Int. J. Sci. Technol. Res.*, vol. 6, pp. 285–290, Feb. 2017. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10156267>
- [51] S. Kimathi, S. Kang'ethe, and P. Kihato, "UAV heading controller using reinforcement learning," in *Proc. Pan Afr. Conf. Sci., Comput. Telecommun. (PACT)*. Nairobi, Kenya: Strathmore Univ., 2017. [Online]. Available: <https://su-plus.strathmore.edu/items/92d45c63-3683-458a-9ee8-00296a615506>
- [52] B. D. Anderson and J. B. Moore, *Optimal Control: Linear Quadratic Methods*. New York, NY, USA: Dover, 2007. [Online]. Available: <https://www.amazon.com/-/en/Brian-D-Anderson/dp/0486457664>
- [53] L. Chrif and Z. M. Kadda, "Aircraft control system using LQG and LQR controller with optimal estimation-Kalman filter design," *Proc. Eng.*, vol. 80, pp. 245–257, Jan. 2014.
- [54] A. I. Abdulla and I. K. Mohammed, "Aircraft pitch control design using LQG controller based on genetic algorithm," *Telkomnika (Telecommun. Comput. Electron. Control)*, vol. 21, no. 2, pp. 409–417, 2023.
- [55] A. Maddi, A. Guessoum, and D. Berkani, "Using linear quadratic Gaussian optimal control for lateral motion of aircraft," *Int. J. Aerosp. Mech. Eng.*, vol. 3, no. 1, pp. 19–23, 2009.
- [56] G. E. Dullerud and F. Paganini, *A Course in Robust Control Theory: A Convex Approach: 36* (Texts in Applied Mathematics). New York, NY, USA: Springer, 1999. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/4118465>
- [57] D. Trotta, G. De Matteis, A. Zavoli, and V. D'Antuono, "Optimal tuning for robust control of a small fixed-wing UAV," in *Proc. AIAA Scitech Forum*, 2021, p. 1057.
- [58] T. Wang, L. Zhang, and Z. Chen, "Robust control for underactuated fixed-wing unmanned aerial vehicles," *Mathematics*, vol. 12, no. 7, p. 1118, Apr. 2024.
- [59] S. Kohno and K. Uchiyama, "Design of robust controller of fixed-wing UAV for transition flight," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, May 2014, pp. 1111–1116.
- [60] L. Wang, *Model Predictive Control System Design and Implementation Using MATLAB*. New York, NY, USA: Springer-Verlag, 2009. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7458179>
- [61] P. W. Gibbens and E. D. Medagoda, "Efficient model predictive control algorithm for aircraft," *J. Guid., Control, Dyn.*, vol. 34, no. 6, pp. 1909–1915, 2011.
- [62] L. Chrif and Z. M. Kadda, "Aircraft control system using model predictive controller," *Telkomnika Indonesian J. Electr. Eng.*, vol. 15, no. 2, pp. 259–269, 2015.
- [63] I. D. Landau, R. Lozano, M. M'Saad, and A. Karimi, *Adaptive Control: Algorithms, Analysis and Applications*, 2nd ed., New York, NY, USA: Springer-Verlag, 2011. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10125578/>
- [64] Z. T. Dydek, A. M. Annaswamy, and E. Lavretsky, "Adaptive control of quadrotor UAVs: A design trade study with flight evaluations," *IEEE Trans. Control Syst. Technol.*, vol. 21, no. 4, pp. 1400–1406, Jul. 2013.
- [65] Y. Yildiz, M. Unel, and A. E. Demirel, "Nonlinear hierarchical control of a quad tilt-wing UAV: An adaptive control approach," *Int. J. Adapt. Control Signal Process.*, vol. 31, no. 9, pp. 1245–1264, 2017.
- [66] R. S. Sutton and A. G. Barto, *Introduction to Reinforcement Learning*, vol. 135. Cambridge, MA, USA: MIT Press, 1998.
- [67] R. S. Sutton and A. G. Barto, "Reinforcement learning," *J. Cognit. Neurosci.*, vol. 11, no. 1, pp. 126–134, 1999.
- [68] H. Hasselt, "Double Q-learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010, pp. 2613–2621.
- [69] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1, pp. 1–7.
- [70] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [71] M. Morales, *Grokking Deep Reinforcement Learning*. Shelter Island, NY, USA: Manning Publications, 2020.
- [72] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine, "Soft actor-critic algorithms and applications," 2018, *arXiv:1812.05905*.
- [73] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [74] P. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," 2017, *arXiv:1706.03741*.
- [75] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," 2019, *arXiv:1909.08593*.
- [76] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano, "Learning to summarize with human feedback," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 3008–3021.
- [77] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
- [78] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge, U.K.: Cambridge Univ. Press, 2022.
- [79] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2024, pp. 1–14.
- [80] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, vol. 39, nos. 3–4, pp. 324–345, 1952.
- [81] O. Lukman. (2019). *Gym X-Plane*. [Online]. Available: https://github.com/adderbyte/GYM_XPLANE_ML
- [82] G. Alleon. (2019). *Gym X-Plane*. [Online]. Available: <https://github.com/galleon/gym-jsbsim>
- [83] F. Eckstein, "Learning to fly—building an autopilot system based on neural networks and reinforcement learning," M.Sc dissertation, Fac. Math. Comput. Sci., FernUniversität Hagen, Hagen, Germany, 2020. [Online]. Available: <https://www.fernuni-hagen.de/english/university/faculties-and-services.shtml>
- [84] D. J. Richter and R. A. Calix, "QPlane: An open-source reinforcement learning toolkit for autonomous fixed wing aircraft simulation," in *Proc. 12th ACM Multimedia Syst. Conf.*, 2021, pp. 261–266.
- [85] A. Quessy. (2021). *Fixedwing-Airsim*. [Online]. Available: <https://github.com/AOS55/Fixedwing-Airsim>
- [86] G. Rennie. (2018). *Gym JSBSim*. [Online]. Available: <https://github.com/Gor-Ren/gym-jsbsim>
- [87] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI gym," 2016, *arXiv:1606.01540*.
- [88] P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, Y. Wu, and P. Zhokhov. (2017). *OpenAI Baselines*. [Online]. Available: <https://github.com/openai/baselines>

- [89] A. Hill, A. Raffin, M. Ernestus, A. Gleave, A. Kanervisto, R. Traore, P. Dhariwal, C. Hesse, O. Klimov, A. Nichol, M. Plappert, A. Radford, J. Schulman, S. Sidor, and Y. Wu. (2018). *Stable Baselines*. [Online]. Available: <https://github.com/hill-a/stable-baselines>
- [90] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [91] J. Berndt, "JSBSim: An open source flight dynamics model in C++," in *Proc. AIAA Modeling Simulation Technol. Conf. Exhibit*, 2004, p. 4923.
- [92] Laminar-Research. (2021). *X-Plane*. [Online]. Available: <https://www.x-plane.com/>
- [93] E. Bøhn. (2019). *Gym-Fixed-Wing*. [Online]. Available: <https://github.com/eivindeb/fixed-wing-gym>
- [94] (2019). *Pyfly—Python Fixed Wing Flight Simulator*. [Online]. Available: <https://github.com/eivindeb/pyfly>
- [95] F. Eckstein. (2020). *Marko-Pilot*. [Online]. Available: <https://github.com/opt12/Markov-Pilot>
- [96] D. J. Richter. (2021). *Qplane Fixed Wing Flight Simulation Environment for Reinforcement Learning*. <https://github.com/JDatPNW/QPlane>
- [97] B. Coconnier, S. McLeod, and A. De Marco. (2011). *JSBSim—An Open Source Flight Dynamics & Control Software Library*. [Online]. Available: <https://github.com/JSBSim-Team/jsbsim>
- [98] A. R. Perry, "The flightgear flight simulator," in *Proc. USENIX Annu. Tech. Conf.*, vol. 686, 2004, pp. 1–13.
- [99] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Proc. Field Service Robot.* Berlin, Germany: Springer, 2018, pp. 621–635. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10122127>
- [100] X. Tassin. (2010). *Geofs*. [Online]. Available: <https://www.geo-fs.com/>
- [101] E. Bøhn, "Reinforcement learning for optimization of nonlinear and predictive control," Philosophiae Doctor Trondheim, Dept. Eng. Cybern., Fac. Inf. Technol. Elect. Eng., Norwegian Univ. Sci. Technol., Mar. 2022. [Online]. Available: https://ntnuopen.ntnu.no/ntnu-xmlui/bitstream/handle/11250/2981909/Eivind%20B%C3%B8hn_PhD.pdf
- [102] E. Bohn, E. M. Coates, D. Reinhardt, and T. A. Johansen, "Data-efficient deep reinforcement learning for attitude control of fixed-wing UAVs: Field experiments," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3168–3180, Mar. 2024.
- [103] D. P. Reinhardt, "On nonlinear and optimization-based control of fixed-wing unmanned aerial vehicles," Philosophiae Doctor Trondheim, Dept. Eng. Cybern., Fac. Inf. Technol. Elect. Eng., Norwegian Univ. Sci. Technol., Jun. 2022. [Online]. Available: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/2998704>
- [104] E. M. Coates, D. Reinhardt, K. Gryte, and T. A. Johansen, "Toward nonlinear flight control for fixed-wing UAVs: System architecture, field experiments, and lessons learned," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2022, pp. 724–734.
- [105] E. M. L. Coates, "Nonlinear attitude and path-following control of fixed-wing aircraft," Philosophiae Doctor Trondheim, Dept. Eng. Cybern., Fac. Inf. Technol. Elect. Eng., Norwegian Univ. Sci. Technol., Sep. 2023. [Online]. Available: <https://ntnuopen.ntnu.no/ntnu-xmlui/handle/11250/3086295>
- [106] J. Monaco, D. Ward, and A. Barto, "Automated aircraft recovery via reinforcement learning: Initial experiments," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 10, 1997, pp. 1022–1028.
- [107] L. Baird, "Residual algorithms: Reinforcement learning with function approximation," in *Machine Learning Proceedings*. Amsterdam, The Netherlands: Elsevier, 1995, pp. 30–37.
- [108] M. E. Harmon and L. C. Baird III, "Multi-player residual advantage learning with general function approximation," Wright Lab. WL/AACF, Wright-Patterson Air Force Base, OH, USA, Tech. Rep. WL-TR-1065, 1996. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9448399>
- [109] J. A. Bagnell and J. G. Schneider, "Autonomous helicopter control using reinforcement learning policy search methods," in *Proc. IEEE Int. Conf. Robot. Automat.*, vol. 2, May 2001, pp. 1615–1620.
- [110] A. Y. Ng, A. Coates, M. Diel, V. Ganapathi, J. Schulte, B. Tse, E. Berger, and E. Liang, "Autonomous inverted helicopter flight via reinforcement learning," in *Experimental Robotics IX*. Berlin, Germany: Springer, 2006, pp. 363–372. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8461249>
- [111] P. Abbeel, A. Coates, M. Quigley, and A. Ng, "An application of reinforcement learning to aerobatic helicopter flight," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 19, 2006, pp. –8.
- [112] N. Bernini, M. Bessa, R. Delmas, A. Gold, E. Goubault, R. Pennec, S. Putot, and F. Sillion, "A few lessons learned in reinforcement learning for quadcopter attitude control," in *Proc. 24th Int. Conf. Hybrid Systems: Comput. Control*, May 2021, pp. 1–11.
- [113] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 2, no. 4, pp. 2096–2103, Oct. 2017.
- [114] E. L. Hayes, "Machine learning for intelligent control: Application of reinforcement learning techniques to the development of flight control systems for miniature UAV rotorcraft," M.S. thesis, Fac. Mech. Eng., Univ. Canterbury, Christchurch, New Zealand, 2013. [Online]. Available: <https://www.canterbury.ac.nz/study/academic-study/engineering/schools-and-departments-engineering-forestry-product-design/mechanical-engineering-department>
- [115] X. Huang, W. Luo, and J. Liu, "Attitude control of fixed-wing UAV based on DDQN," in *Proc. Chin. Autom. Congr. (CAC)*, Nov. 2019, pp. 4722–4726.
- [116] D. Wada, S. A. Araujo-Estrada, and S. Windsor, "Unmanned aerial vehicle pitch control using deep reinforcement learning with discrete actions in wind tunnel test," *Aerospace*, vol. 8, no. 1, p. 18, Jan. 2021.
- [117] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [118] M. Zahmatkesh, S. Emami, A. Banazadeh, and P. Castaldi, "Robust attitude control of an agile aircraft using improved Q-learning," *Actuators*, vol. 11, no. 12, p. 374, Dec. 2022.
- [119] M. Chowdhury and S. Keshmiri, "Design and flight test validation of an AI-based longitudinal flight controller for fixed-wing UAVs," in *Proc. IEEE Aerosp. Conf. (AERO)*, Mar. 2022, pp. 1–12.
- [120] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, "ROS: An open-source robot operating system," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA) Workshop Open Source Robot.*, Kobe, Japan, May 2009, pp. 1–6.
- [121] N. Koenig and A. Howard, "Design and use paradigms for gazebo, an open-source multi-robot simulator," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, vol. 3, Sep./Oct. 2004, pp. 2149–2154.
- [122] D. J. Richter, L. Natonski, X. Shen, and R. A. Calix, "Attitude control for fixed-wing aircraft using Q-learning," in *Intelligent Human Computer Interaction (Lecture Notes in Computer Science)*, vol. 13184, J. H. Kim, M. Singh, J. Khan, U. S. Tiwary, M. Sur, and D. Singh, Eds., Cham, Switzerland: Springer, 2022, doi: [10.1007/978-3-030-98404-5_59](https://doi.org/10.1007/978-3-030-98404-5_59). [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-030-98404-5_59#citeas
- [123] D. J. Richter, "Simulated fixed-wing aircraft attitude control using reinforcement learning methods," Ph.D. dissertation, Purdue Univ. Graduate School, West Lafayette, IN, USA, 2021.
- [124] D. J. Richter and R. A. Calix, "Using double deep Q-learning to learn attitude control of fixed-wing aircraft," in *Proc. 16th Int. Conf. Signal-Image Technol. Internet-Based Syst. (SITIS)*, Oct. 2022, pp. 646–651.
- [125] X. Li, J. Wu, C. Qi, and P. Cong, "Deep reinforcement learning and L1 adaptive control algorithm-based attitude control of fixed-wing UAVs," in *Proc. Int. Conf. Auton. Unmanned Syst.* Singapore: Springer, 2022, pp. 2273–2285. [Online]. Available: https://link.springer.com/chapter/10.1007/978-981-99-0479-2_212
- [126] N. Hovakimyan and C. Cao, *L1 Adaptive Control Theory: Guaranteed Robustness With Fast Adaptation*. Philadelphia, PA, USA: SIAM, 2010.
- [127] X. Qiu, C. Gao, K. Wang, and W. Jing, "Attitude control of a moving mass-actuated UAV based on deep reinforcement learning," *J. Aerosp. Eng.*, vol. 35, no. 2, Mar. 2022, Art. no. 04021133.
- [128] M. Chowdhury and S. Keshmiri, "Interchangeable reinforcement-learning flight controller for fixed-wing UAVs," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 2, pp. 2305–2318, Apr. 2024.
- [129] G. A. Rummery and M. Niranjan, *On-Line Q-Learning Using Connectionist Systems*, vol. 37. Cambridge, U.K.: University Cambridge, 1994.
- [130] R. Lowe, Y. I. Wu, A. A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–12.
- [131] S. Zhang, X. Du, J. Xiao, J. Huang, and K. He, "Reinforcement learning control for 6 DOF flight of fixed-wing aircraft," in *Proc. 33rd Chin. Control Decis. Conf. (CCDC)*, May 2021, pp. 5454–5460.
- [132] C. Yan and X. Xiang, "A path planning algorithm for UAV based on improved Q-learning," in *Proc. 2nd Int. Conf. Robot. Autom. Sci. (ICRAS)*, Jun. 2018, pp. 1–5.

- [133] C. Yan, X. Xiang, and C. Wang, "Towards real-time path planning through deep reinforcement learning for a UAV in dynamic environments," *J. Intell. Robot. Syst.*, vol. 98, pp. 297–309, Sep. 2020.
- [134] Y. Zhou, J. Shu, H. Hao, H. Song, and X. Lai, "UAV 3D online track planning based on improved SAC algorithm," *J. Brazilian Soc. Mech. Sci. Eng.*, vol. 46, no. 1, p. 12, Jan. 2024.
- [135] S.-M. Hung, S. N. Givigi, and A. Noureldin, "A Dyna-Q (Lambda) approach to flocking with fixed-wing UAVs in a stochastic environment," in *Proc. IEEE Int. Conf. Syst., Man, Cybern.*, Oct. 2015, pp. 1918–1923.
- [136] S.-M. Hung and S. N. Givigi, "A Q-learning approach to flocking with UAVs in a stochastic environment," *IEEE Trans. Cybern.*, vol. 47, no. 1, pp. 186–197, Jan. 2017.
- [137] J. Peng and R. J. Williams, "Incremental multi-step Q-learning," in *Machine Learning Proceedings*. Amsterdam, The Netherlands: Elsevier, 1994, pp. 226–232.
- [138] S. A. P. Quintero, G. E. Collins, and J. P. Hespanha, "Flocking with fixed-wing UAVs for distributed sensing: A stochastic optimal control approach," in *Proc. Amer. Control Conf.*, Jun. 2013, pp. 2025–2031.
- [139] C. Wang, C. Yan, X. Xiang, and H. Zhou, "A continuous actor-critic reinforcement learning approach to flocking with fixed-wing UAVs," in *Proc. Asian Conf. Mach. Learn.*, 2019, pp. 64–79.
- [140] C. Yan, X. Xiang, and C. Wang, "Fixed-wing UAVs flocking in continuous spaces: A deep reinforcement learning approach," *Robot. Auto. Syst.*, vol. 131, Sep. 2020, Art. no. 103594.
- [141] H. Van Hasselt and M. A. Wiering, "Reinforcement learning in continuous action spaces," in *Proc. IEEE Int. Symp. Approx. Dyn. Program. Reinforcement Learn.*, Apr. 2007, pp. 272–279.
- [142] H. van Hasselt and M. A. Wiering, "Using continuous action spaces to solve discrete problems," in *Proc. Int. Joint Conf. Neural Netw.*, Jun. 2009, pp. 1149–1156.
- [143] C. Yan, X. Xiang, C. Wang, and Z. Lan, "Flocking and collision avoidance for a dynamic squad of fixed-wing UAVs using deep reinforcement learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2021, pp. 4738–4744.
- [144] C. Yan, C. Wang, X. Xiang, Z. Lan, and Y. Jiang, "Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing UAVs using local situation maps," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1260–1270, Feb. 2022.
- [145] C. Yan, C. Wang, X. Xiang, K. H. Low, X. Wang, X. Xu, and L. Shen, "Collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments: A task-specific curriculum-based MADRL approach," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Feb. 23, 2023, doi: 10.1109/TNNLS.2023.3245124. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10051636>
- [146] D. Xu, Y. Guo, Z. Yu, Z. Wang, R. Lan, R. Zhao, X. Xie, and H. Long, "PPO-exp: Keeping fixed-wing UAV formation with deep reinforcement learning," *Drones*, vol. 7, no. 1, p. 28, Dec. 2022.
- [147] X. Yuan, Y. Sun, Y. Wang, and C. Sun, "Deterministic policy gradient with advantage function for fixed wing UAV automatic landing," in *Proc. Chin. Control Conf. (CCC)*, Jul. 2019, pp. 8305–8310.
- [148] C. Tang and Y.-C. Lai, "Deep reinforcement learning automatic landing control of fixed-wing aircraft using deep deterministic policy gradient," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Sep. 2020, pp. 1–9.
- [149] M. Zahmatkesh, S. Ali Emami, A. Banazadeh, and P. Castaldi, "Robust auto-landing control of an agile regional jet using fuzzy Q-learning," 2023, *arXiv:2302.10997*.
- [150] P. Y. Glorennec and L. Jouffe, "Fuzzy Q-learning," in *Proc. 6th Int. Fuzzy Syst. Conf.*, vol. 2, Jul. 1997, pp. 659–662.
- [151] L. J. Fletcher, R. J. Clarke, T. S. Richardson, and M. Hansen, "Reinforcement learning for a perched landing in the presence of wind," in *Proc. AIAA Scitech Forum*, 2021, p. 1282.
- [152] L. J. Fletcher, R. J. Clarke, T. S. Richardson, and M. Hansen, "Integrating throttle into a reinforcement learning controller for a perched landing of a variable sweep wing UAV," in *Proc. AIAA SCITECH Forum*, 2022, p. 1288.
- [153] H. Zhang, Z. Jiao, Y. Shang, X. Liu, P. Qi, and S. Wu, "Ground maneuver for front-wheel drive aircraft via deep reinforcement learning," *Chin. J. Aeronaut.*, vol. 34, no. 10, pp. 166–176, 2021.
- [154] Z. Xiang, H. Sun, and J. Zhang, "Application of improved Q-learning algorithm in dynamic path planning for aircraft at airports," *IEEE Access*, vol. 11, pp. 107892–107905, 2023.
- [155] W. Chang, W. Lizhen, Y. Chao, W. Zhichao, L. Han, and Y. Chao, "Coactive design of explainable agent-based task planning and deep reinforcement learning for human-UAVs teamwork," *Chin. J. Aeronaut.*, vol. 33, no. 11, pp. 2930–2945, 2020.
- [156] F. d'Apolito, "Reinforcement learning training environment for fixed wing UAV collision avoidance," *IFAC-PapersOnLine*, vol. 55, no. 39, pp. 281–285, 2022.
- [157] W. Hu, Z. Gao, J. Quan, X. Ma, J. Xiong, and W. Zhang, "Fixed-wing stalled maneuver control technology based on deep reinforcement learning," in *Proc. IEEE 5th Int. Conf. Big Data Artif. Intell. (BDAI)*, Jul. 2022, pp. 19–25.
- [158] Y. Ou, H. Jiang, Z. Xu, W. Lu, and H. Xiong, "Reinforcement learning-based hierarchical obstacle avoidance strategy for fixed-wing aircraft," in *Proc. China Autom. Congr. (CAC)*, China, Nov. 2022, pp. 1815–1819.
- [159] J. Wu, H. Wang, Y. Liu, M. Zhang, and T. Wu, "Learning-based fixed-wing UAV reactive maneuver control for obstacle avoidance," *Aerosp. Sci. Technol.*, vol. 126, Jul. 2022, Art. no. 107623.
- [160] J. Choi, H. M. Kim, H. J. Hwang, Y.-D. Kim, and C. O. Kim, "Modular reinforcement learning for autonomous UAV flight control," *Drones*, vol. 7, no. 7, p. 418, Jun. 2023.
- [161] L. Li, X. Zhang, C. Qian, and R. Wang, "Basic flight maneuver generation of fixed-wing plane based on proximal policy optimization," *Neural Comput. Appl.*, vol. 35, no. 14, pp. 10239–10255, May 2023.
- [162] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. T. J. Shen, and O. G. Younis. (Mar. 2023). *Gymnasium*. [Online]. Available: <https://zenodo.org/record/8127025>



DAVID J. RICHTER received the B.S. degree in computer science from Kempten University of Applied Sciences, Germany, in 2019, and the M.S. degree in computer information technology from Purdue University Northwest, Hammond, IN, USA, in 2021. He is currently pursuing the master's degree in artificial intelligence convergence with Chonnam National University, Gwangju, South Korea. He was a Teaching and Research Assistant with the College of Technology, Purdue University Northwest, and a Research Assistant with Kempten University of Applied Sciences. He is also a Research Assistant with Chonnam National University. His research interests include AI, machine and deep learning, reinforcement learning, genetic programming, and study abroad.



RICARDO A. CALIX received the B.S. degree in industrial and systems engineering from Universidad Tecnológica Centroamericana, Tegucigalpa, Honduras, in 2001, the master's degree from Louisiana State University (LSU), Baton Rouge, in 2006, and the Ph.D. degree in engineering science with a research focus on natural language processing and machine learning from LSU, in 2011. From 2007 to 2011, he was a Graduate Assistant with the Department of Industrial Engineering, LSU, where he conducted research in natural language processing. He is currently a Full Professor of Computer Information Technology at Purdue University Northwest, and an Independent Data Science Consultant. His research interests include natural language processing, deep learning, and reinforcement learning.



KYUNGBAEK KIM (Member, IEEE) received the B.S., M.S., and Ph.D. degrees in electrical engineering and computer science from Korea Advanced Institute of Science and Technology (KAIST), South Korea, in 1999, 2001, and 2007, respectively. He is currently a Professor with the Department of Artificial Intelligence Convergence, Chonnam National University. Previously, he was a Postdoctoral Researcher with the Department of Computer Sciences, University of California, Irvine, CA, USA. His research interests include intelligent distributed systems, software-defined networks/infrastructure, big data platform, GRID/cloud systems, social networking systems, AI applied cyber-physical systems, blockchain, and other issues of distributed systems. He is a member of ACM, IEICE, KIISE, KIPS, KICS, KIISC, and KISM.

...